

Dimensionality Reduction

by Lynn Njoroge

Defining the Question

Specifying the Question

You are a Data analyst at Carrefour Kenya and are currently undertaking a project that will inform the marketing department on the most relevant marketing strategies that will result in the highest no. of sales (total price including tax). Perform dimensionality reduction techniques on the dataset using the t-SNE algorithm or PCA and provide recommendations based on your insights.

Defining the Metric for Success

Our project will be considered successful if we are able to reduce the variables in our dataset to a low dimensional dataset and provide insights gained from your analysis.

3. Understanding the context

The provided data is from Carrefour Kenya's database and shows transactions that have been underway over a certain period. This is a reflection of behaviour of sales at Carrefour and we will use it to identify principal components in the transactions.

Recording the Experimental Design

Defining the Question Reading the Data. Checking the Data. Data Cleaning Performing EDA Implementing PCA Implementing Feature Selection Conclusion

Data Relevance

This will be discussed after the analysis and prediction has been completed.

Data Preparation

Importing Necessary Libraries

```
library(tidyverse)
```

```
## -- Attaching packages ----- tidyverse 1.3.0 --
```

```
## v ggplot2 3.3.3    v purrr   0.3.4
## v tibble  3.1.0    v dplyr   1.0.5
## v tidyr   1.1.3    v stringr 1.4.0
## v readr   1.4.0    v forcats 0.5.1
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag() masks stats::lag()
```

```
library(ggplot2)
library(caret)
```

```
## Loading required package: lattice
```

```
##
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
##
## lift
```

```
library(caretEnsemble)
```

```
##
## Attaching package: 'caretEnsemble'
```

```
## The following object is masked from 'package:ggplot2':
##
## autoplot
```

```
library(psych)
```

```
##
## Attaching package: 'psych'
```

```
## The following objects are masked from 'package:ggplot2':
##
## %+%, alpha
```

```
library(Amelia)
```

```
## Loading required package: Rcpp
```

```
## ##
## ## Amelia II: Multiple Imputation
## ## (Version 1.7.6, built: 2019-11-24)
## ## Copyright (C) 2005-2021 James Honaker, Gary King and Matthew Blackwell
## ## Refer to http://gking.harvard.edu/amelia/ for more information
## ##
```

```
library(mice)
```

```
##
## Attaching package: 'mice'
```

```
## The following object is masked from 'package:stats':  
##  
##   filter
```

```
## The following objects are masked from 'package:base':  
##  
##   cbind, rbind
```

```
library(GGally)
```

```
## Registered S3 method overwritten by 'GGally':  
##   method from  
##   +.gg   ggplot2
```

```
library(rpart)  
library(devtools)
```

```
## Loading required package: usethis
```

```
## Warning: package 'usethis' was built under R version 4.0.5
```

```
library(ggbiplot)
```

```
## Loading required package: plyr
```

```
## -----
```

```
## You have loaded plyr after dplyr - this is likely to cause problems.  
## If you need functions from both plyr and dplyr, please load plyr first, then dplyr:  
## library(plyr); library(dplyr)
```

```
## -----
```

```
##  
## Attaching package: 'plyr'
```

```
## The following objects are masked from 'package:dplyr':  
##  
##   arrange, count, desc, failwith, id, mutate, rename, summarise,  
##   summarize
```

```
## The following object is masked from 'package:purrr':  
##  
##   compact
```

```
## Loading required package: scales
```

```
##  
## Attaching package: 'scales'
```

```
## The following objects are masked from 'package:psych':
##
##   alpha, rescale

## The following object is masked from 'package:purrr':
##
##   discard

## The following object is masked from 'package:readr':
##
##   col_factor

## Loading required package: grid
```

Loading and Previewing the data

```
# Loading the Dataset from the source i.e. csv
df <- read.csv("C:/Users/LENOVO/Downloads/Supermarket_Dataset_1 - Sales Data.csv")

head(df)
```

```
##   Invoice.ID Branch Customer.type Gender      Product.line Unit.price
## 1 750-67-8428      A      Member Female Health and beauty      74.69
## 2 226-31-3081      C      Normal Female Electronic accessories  15.28
## 3 631-41-3108      A      Normal  Male   Home and lifestyle    46.33
## 4 123-19-1176      A      Member  Male   Health and beauty    58.22
## 5 373-73-7910      A      Normal  Male   Sports and travel     86.31
## 6 699-14-3026      C      Normal  Male   Electronic accessories  85.39
##   Quantity      Tax      Date Time      Payment      cogs gross.margin.percentage
## 1         7 26.1415 1/5/2019 13:08      Ewallet 522.83      4.761905
## 2         5  3.8200 3/8/2019 10:29      Cash 76.40      4.761905
## 3         7 16.2155 3/3/2019 13:23 Credit card 324.31      4.761905
## 4         8 23.2880 1/27/2019 20:33      Ewallet 465.76      4.761905
## 5         7 30.2085 2/8/2019 10:37      Ewallet 604.17      4.761905
## 6         7 29.8865 3/25/2019 18:30      Ewallet 597.73      4.761905
##   gross.income Rating      Total
## 1      26.1415      9.1 548.9715
## 2       3.8200      9.6  80.2200
## 3      16.2155      7.4 340.5255
## 4      23.2880      8.4 489.0480
## 5      30.2085      5.3 634.3785
## 6      29.8865      4.1 627.6165
```

Checking the Data

```
#checking out all the variables in the data set.
names(df)
```

```
## [1] "Invoice.ID"          "Branch"
## [3] "Customer.type"        "Gender"
## [5] "Product.line"         "Unit.price"
## [7] "Quantity"             "Tax"
## [9] "Date"                 "Time"
## [11] "Payment"              "cogs"
## [13] "gross.margin.percentage" "gross.income"
## [15] "Rating"               "Total"
```

```
#checking the shape of the data
```

```
# checking the dimensions of the data (number of entries and fields)
```

```
dim(df)
```

```
## [1] 1000 16
```

```
#understanding the structure of the data set, data type of each attribute and number of rows and columns
```

```
str(df)
```

```
## 'data.frame': 1000 obs. of 16 variables:
## $ Invoice.ID : chr "750-67-8428" "226-31-3081" "631-41-3108" "123-19-1176" ...
## $ Branch : chr "A" "C" "A" "A" ...
## $ Customer.type : chr "Member" "Normal" "Normal" "Member" ...
## $ Gender : chr "Female" "Female" "Male" "Male" ...
## $ Product.line : chr "Health and beauty" "Electronic accessories" "Home and lifestyle" ...
## $ Unit.price : num 74.7 15.3 46.3 58.2 86.3 ...
## $ Quantity : int 7 5 7 8 7 7 6 10 2 3 ...
## $ Tax : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Date : chr "1/5/2019" "3/8/2019" "3/3/2019" "1/27/2019" ...
## $ Time : chr "13:08" "10:29" "13:23" "20:33" ...
## $ Payment : chr "Ewallet" "Cash" "Credit card" "Ewallet" ...
## $ cogs : num 522.8 76.4 324.3 465.8 604.2 ...
## $ gross.margin.percentage: num 4.76 4.76 4.76 4.76 4.76 ...
## $ gross.income : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Rating : num 9.1 9.6 7.4 8.4 5.3 4.1 5.8 8 7.2 5.9 ...
## $ Total : num 549 80.2 340.5 489 634.4 ...
```

```
#summarising each attribute in the dataset
```

```
summary(df)
```

```
## Invoice.ID      Branch      Customer.type      Gender
## Length:1000    Length:1000    Length:1000      Length:1000
## Class :character Class :character Class :character  Class :character
## Mode :character Mode :character  Mode :character  Mode :character
##
##
## Product.line    Unit.price    Quantity      Tax
## Length:1000     Min. :10.08   Min. : 1.00    Min. : 0.5085
## Class :character 1st Qu.:32.88 1st Qu.: 3.00  1st Qu.: 5.9249
## Mode :character  Median :55.23 Median : 5.00  Median :12.0880
##                  Mean :55.67  Mean : 5.51   Mean :15.3794
##                  3rd Qu.:77.94 3rd Qu.: 8.00  3rd Qu.:22.4453
```

```
##           Max.      :99.96   Max.      :10.00   Max.      :49.6500
##      Date           Time           Payment           cogs
## Length:1000      Length:1000      Length:1000      Min.      : 10.17
## Class :character  Class :character  Class :character  1st Qu.:118.50
## Mode  :character  Mode  :character  Mode  :character  Median :241.76
##                                           Mean  :307.59
##                                           3rd Qu.:448.90
##                                           Max.   :993.00
## gross.margin.percentage gross.income      Rating      Total
## Min.      :4.762      Min.      : 0.5085   Min.      : 4.000   Min.      : 10.68
## 1st Qu.:4.762      1st Qu.: 5.9249   1st Qu.: 5.500   1st Qu.: 124.42
## Median :4.762      Median :12.0880   Median : 7.000   Median : 253.85
## Mean    :4.762      Mean    :15.3794   Mean    : 6.973   Mean    : 322.97
## 3rd Qu.:4.762      3rd Qu.:22.4453   3rd Qu.: 8.500   3rd Qu.: 471.35
## Max.    :4.762      Max.    :49.6500   Max.    :10.000   Max.    :1042.65
```

Data Cleaning

Dealing with Missing Values

```
# let us remove the gross.margin.percentage since all the values are the same
df$gross.margin.percentage <- NULL
```

```
# find out total missing values in each column
```

```
colSums(is.na(df))
```

```
## Invoice.ID      Branch Customer.type      Gender Product.line
##           0           0           0           0           0
## Unit.price      Quantity      Tax      Date      Time
##           0           0           0           0           0
## Payment      cogs gross.income      Rating      Total
##           0           0           0           0           0
```

There are no missing values in our dataset

```
# let us eliminate the missing values
```

```
df1 <- na.omit(df)
colSums(is.na(df1))
```

```
## Invoice.ID      Branch Customer.type      Gender Product.line
##           0           0           0           0           0
## Unit.price      Quantity      Tax      Date      Time
##           0           0           0           0           0
## Payment      cogs gross.income      Rating      Total
##           0           0           0           0           0
```

Dealing with Duplicated values

```
# Now lets find the duplicated rows in the dataset
```

```
duplicated_rows <- df[duplicated(df),]
```

```
duplicated_rows
```

```
## [1] Invoice.ID      Branch          Customer.type  Gender          Product.line  
## [6] Unit.price      Quantity        Tax            Date            Time  
## [11] Payment         cogs            gross.income   Rating          Total  
## <0 rows> (or 0-length row.names)
```

Exploratory Data Analysis

Univariate Analysis

```
# let us select only numerical columns
```

```
nums <- df %>% select_if(is.numeric)  
names(nums)
```

```
## [1] "Unit.price"  "Quantity"    "Tax"          "cogs"         "gross.income"  
## [6] "Rating"      "Total"
```

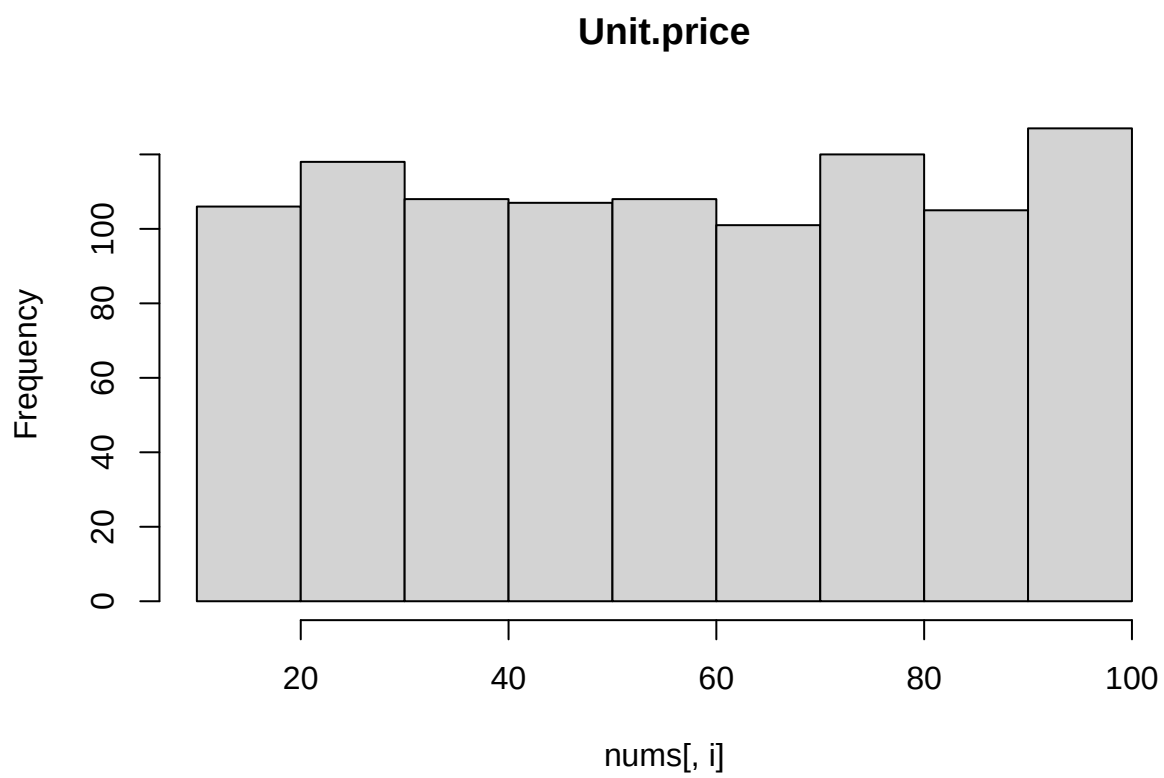
```
# finding the summary statistics of the numerical columns
```

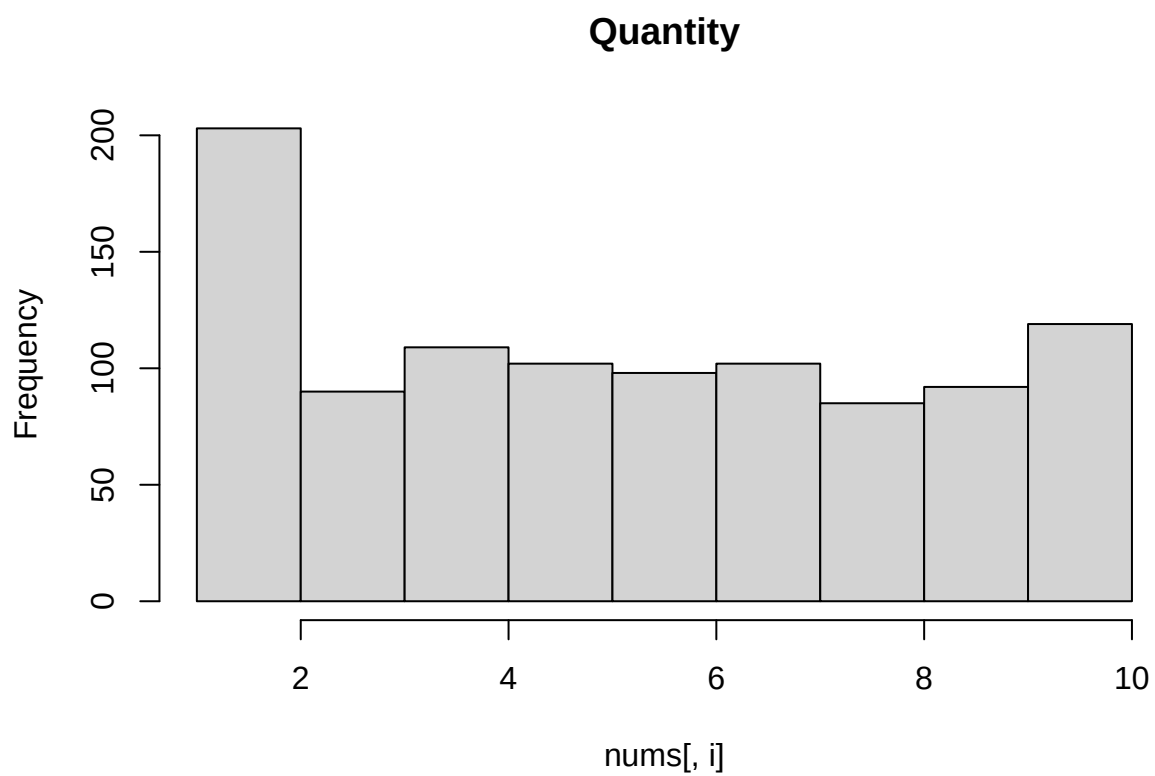
```
summary(nums)
```

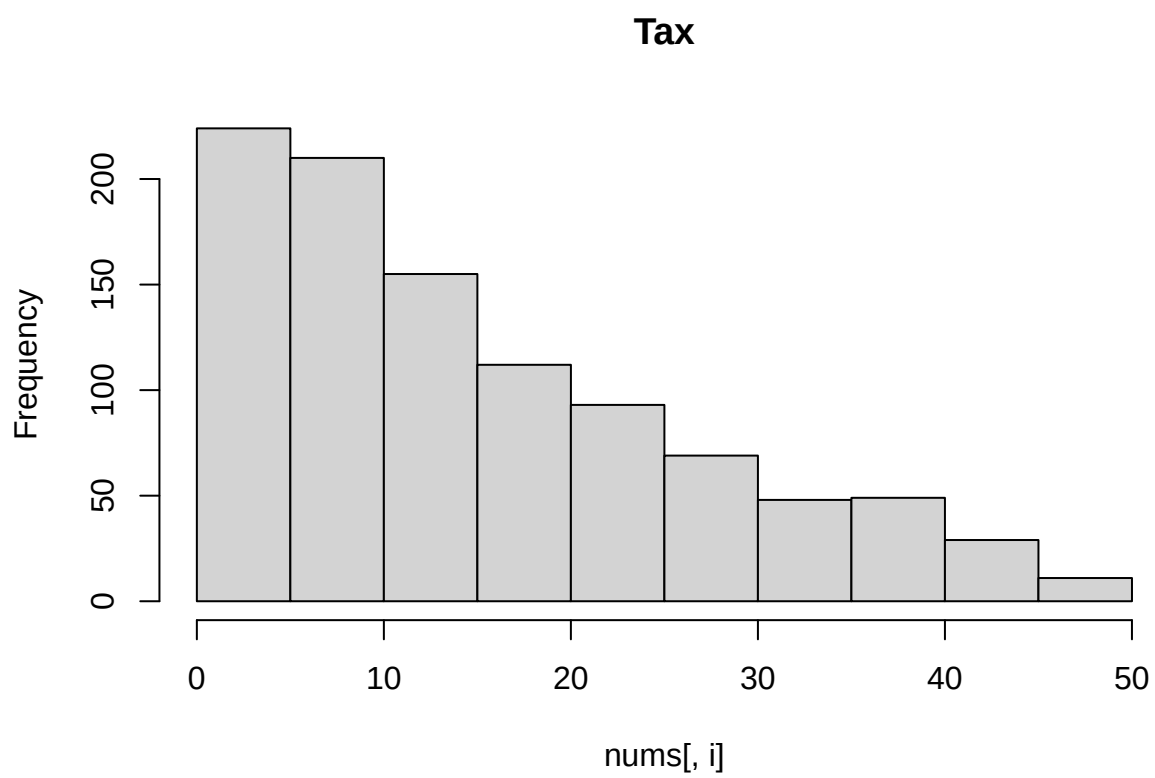
```
##      Unit.price      Quantity      Tax      cogs  
## Min.   :10.08   Min.   : 1.00   Min.   : 0.5085   Min.   : 10.17  
## 1st Qu.:32.88   1st Qu.: 3.00   1st Qu.: 5.9249   1st Qu.:118.50  
## Median :55.23   Median : 5.00   Median :12.0880   Median :241.76  
## Mean   :55.67   Mean   : 5.51   Mean   :15.3794   Mean   :307.59  
## 3rd Qu.:77.94   3rd Qu.: 8.00   3rd Qu.:22.4453   3rd Qu.:448.90  
## Max.   :99.96   Max.   :10.00   Max.   :49.6500   Max.   :993.00  
## gross.income      Rating      Total  
## Min.   : 0.5085   Min.   : 4.000   Min.   : 10.68  
## 1st Qu.: 5.9249   1st Qu.: 5.500   1st Qu.: 124.42  
## Median :12.0880   Median : 7.000   Median : 253.85  
## Mean   :15.3794   Mean   : 6.973   Mean   : 322.97  
## 3rd Qu.:22.4453   3rd Qu.: 8.500   3rd Qu.: 471.35  
## Max.   :49.6500   Max.   :10.000   Max.   :1042.65
```

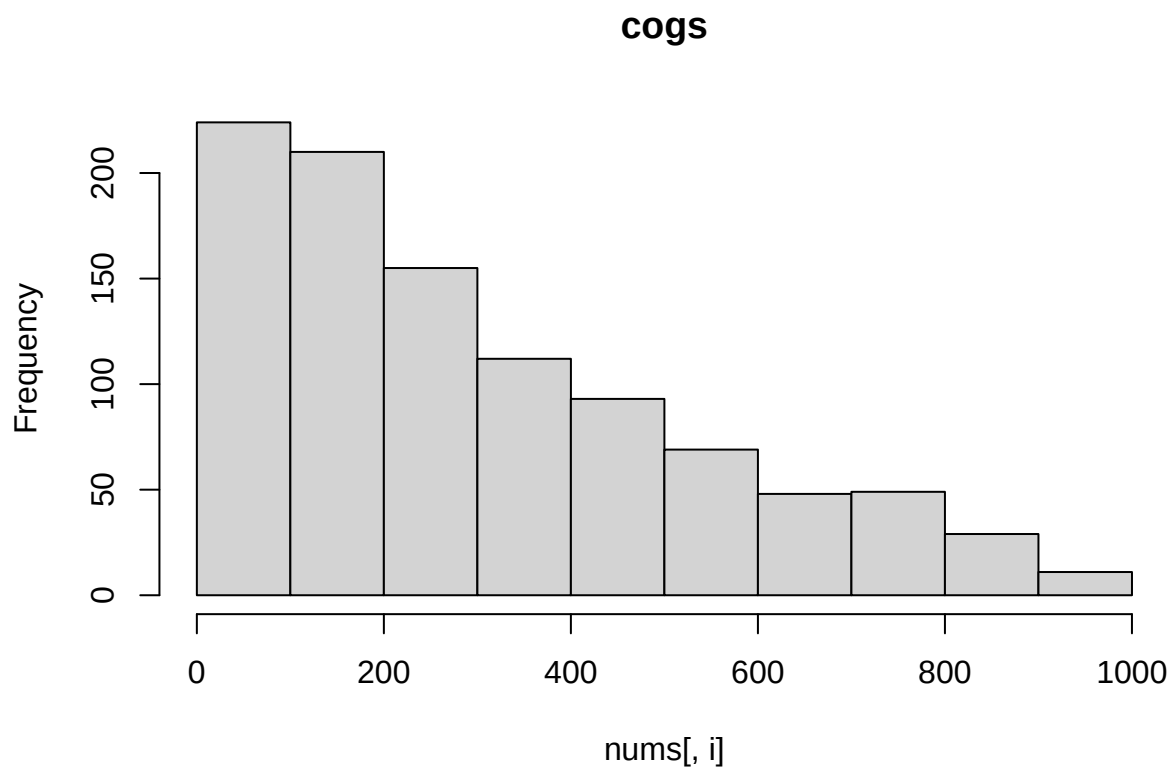
```
#Histograms
```

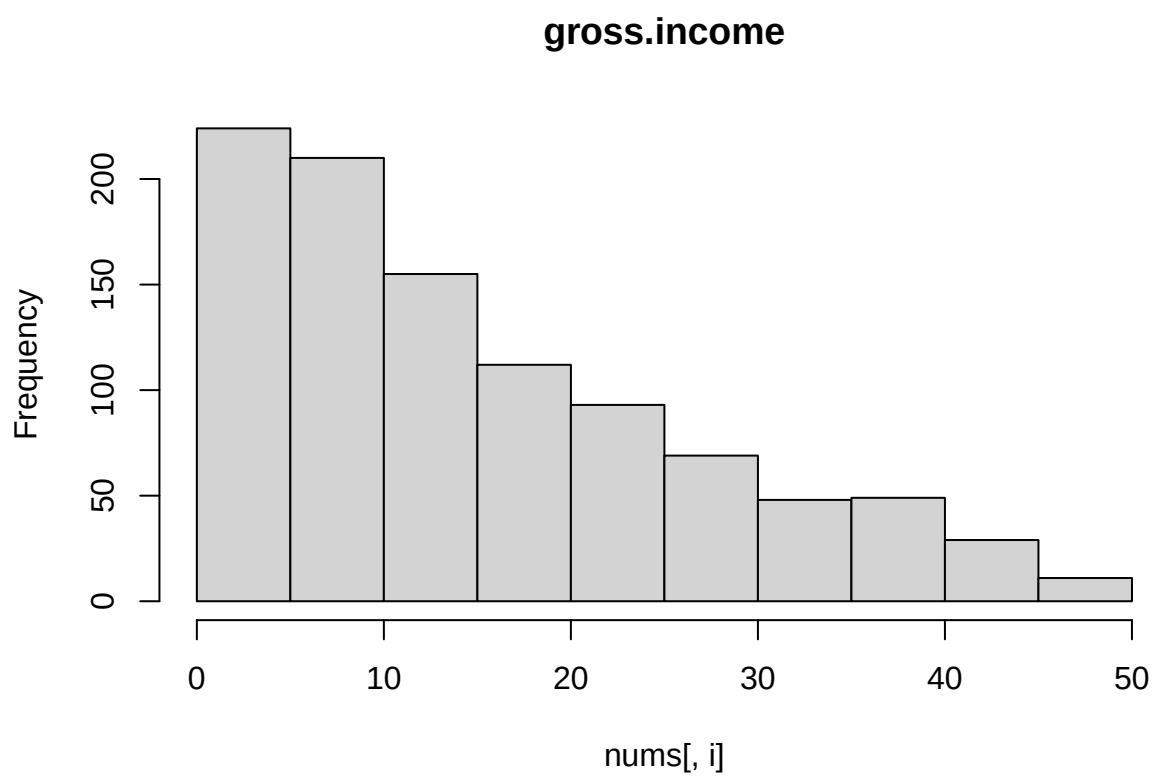
```
for(i in 1 : length(nums)) {  
  hist(nums[,i], main= names(nums[i]))  
}
```

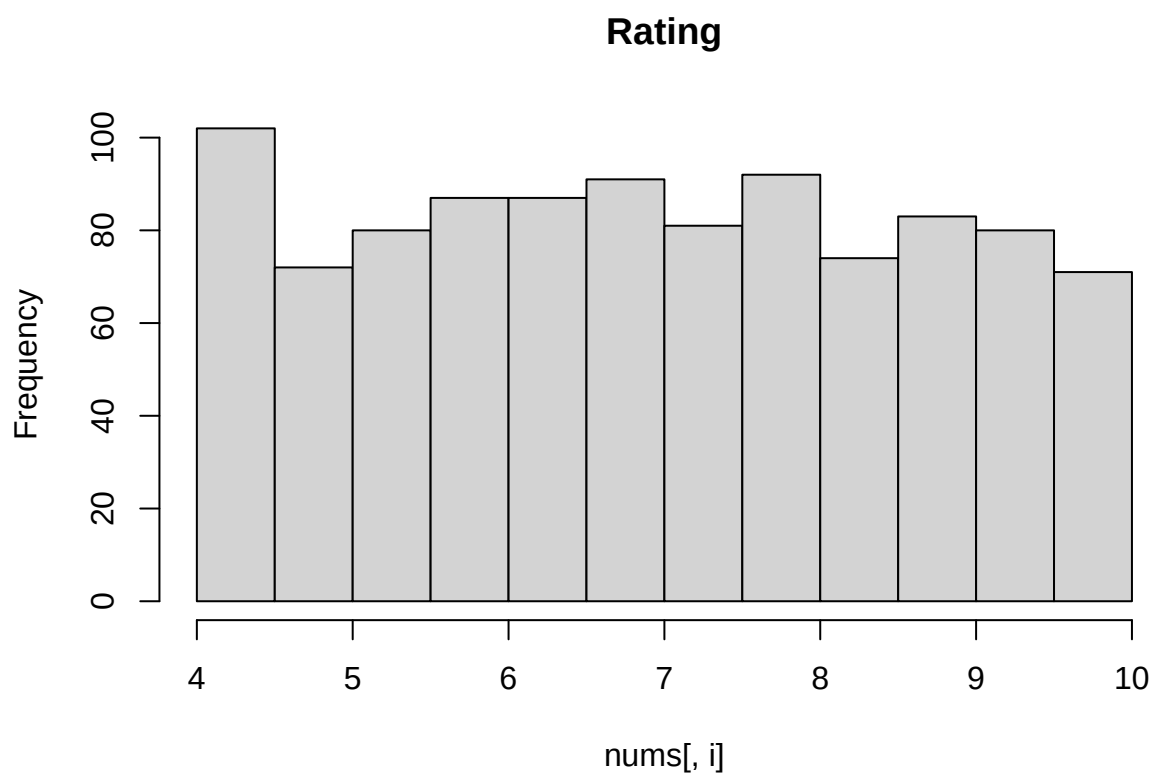


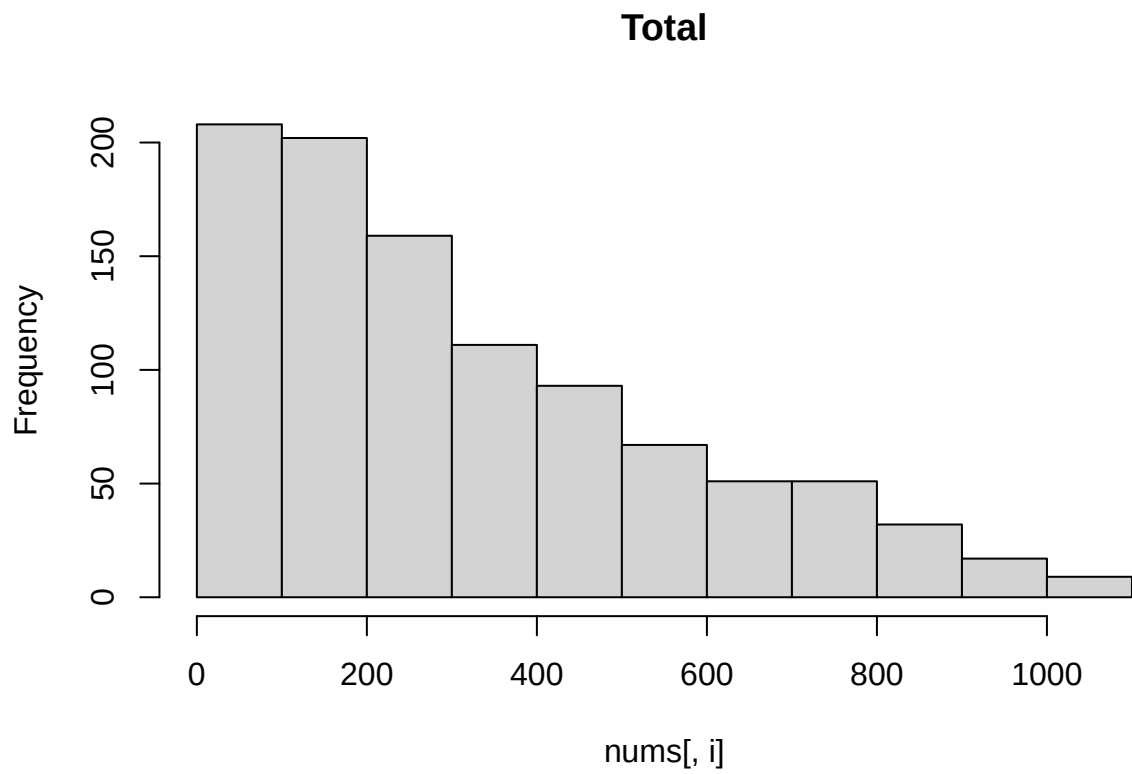






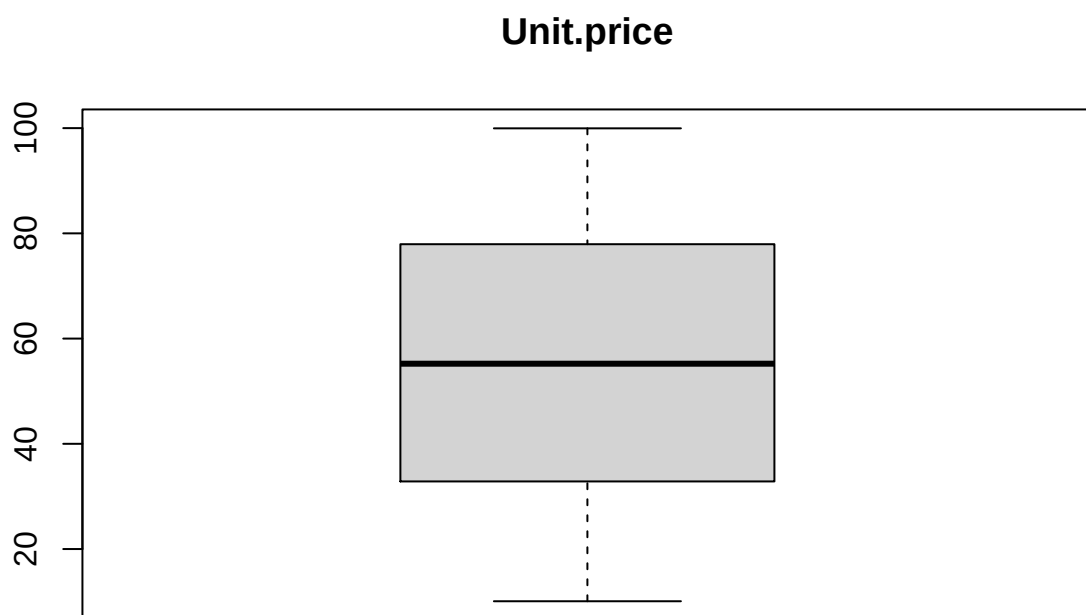


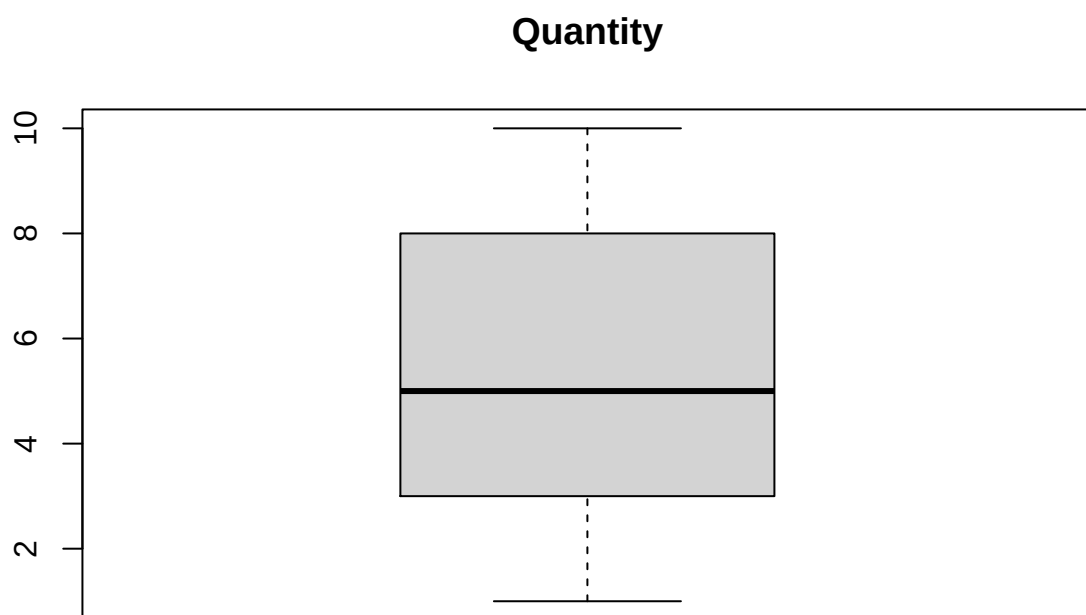


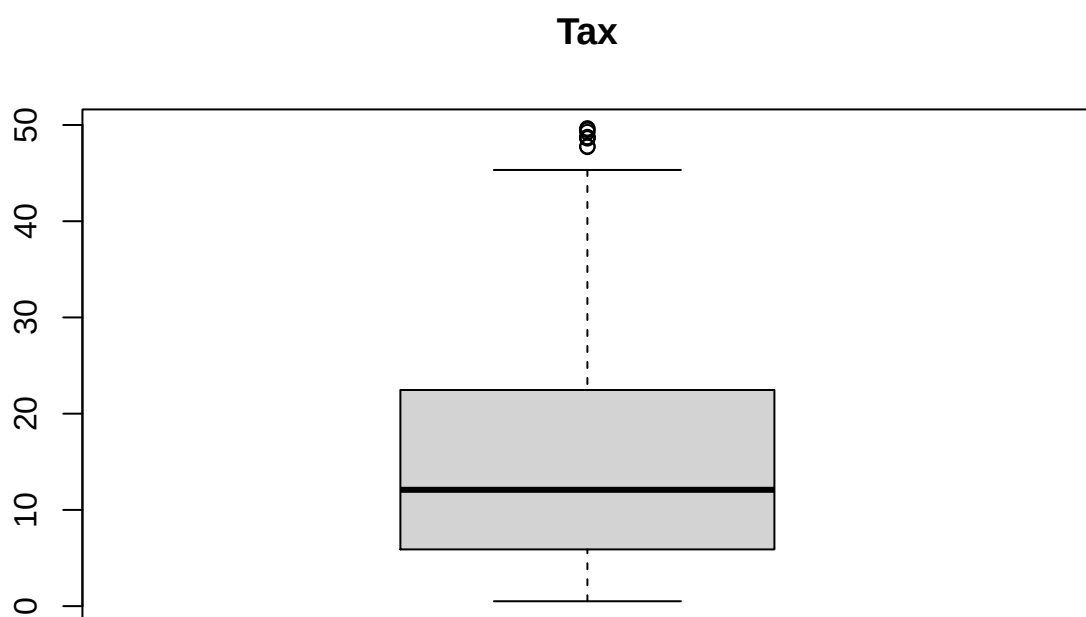


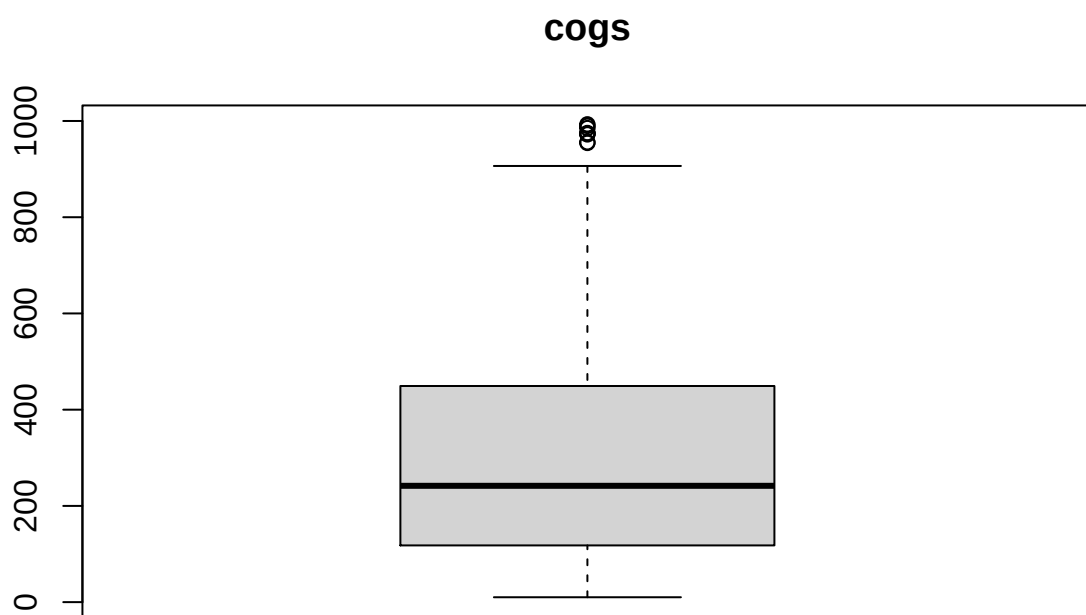
```
# Boxplot for the advertising dataset
```

```
for (i in 1 : length (nums)) {  
  boxplot (nums[,i], main= names (nums[i]), type= "l" )  
}
```

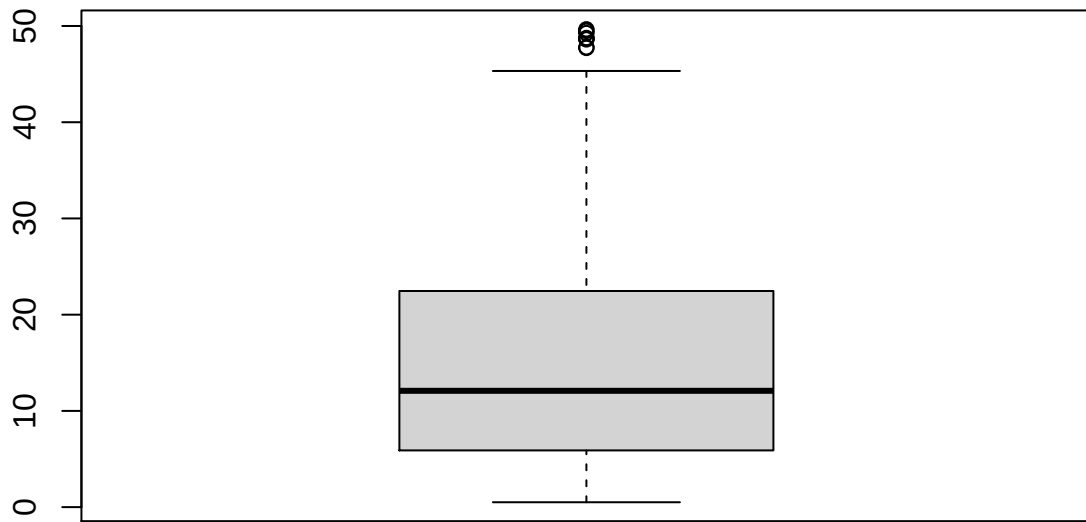


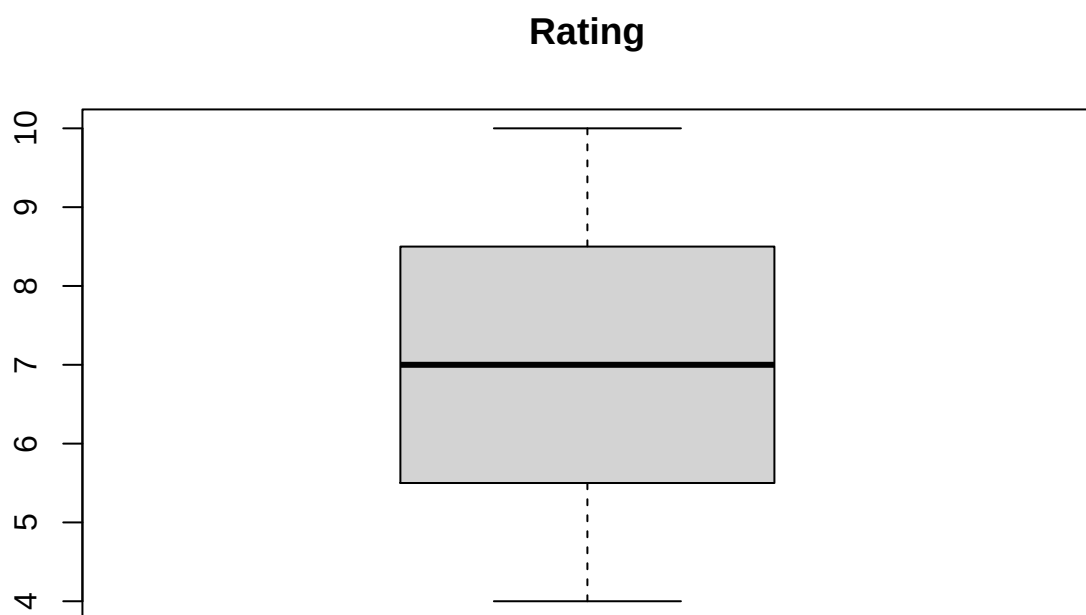


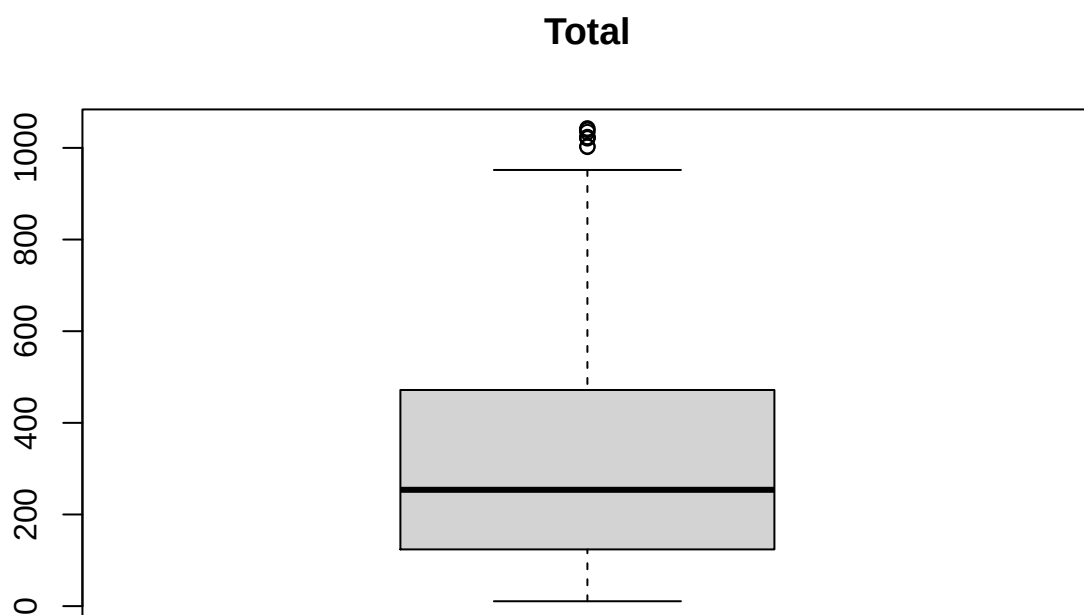




gross.income

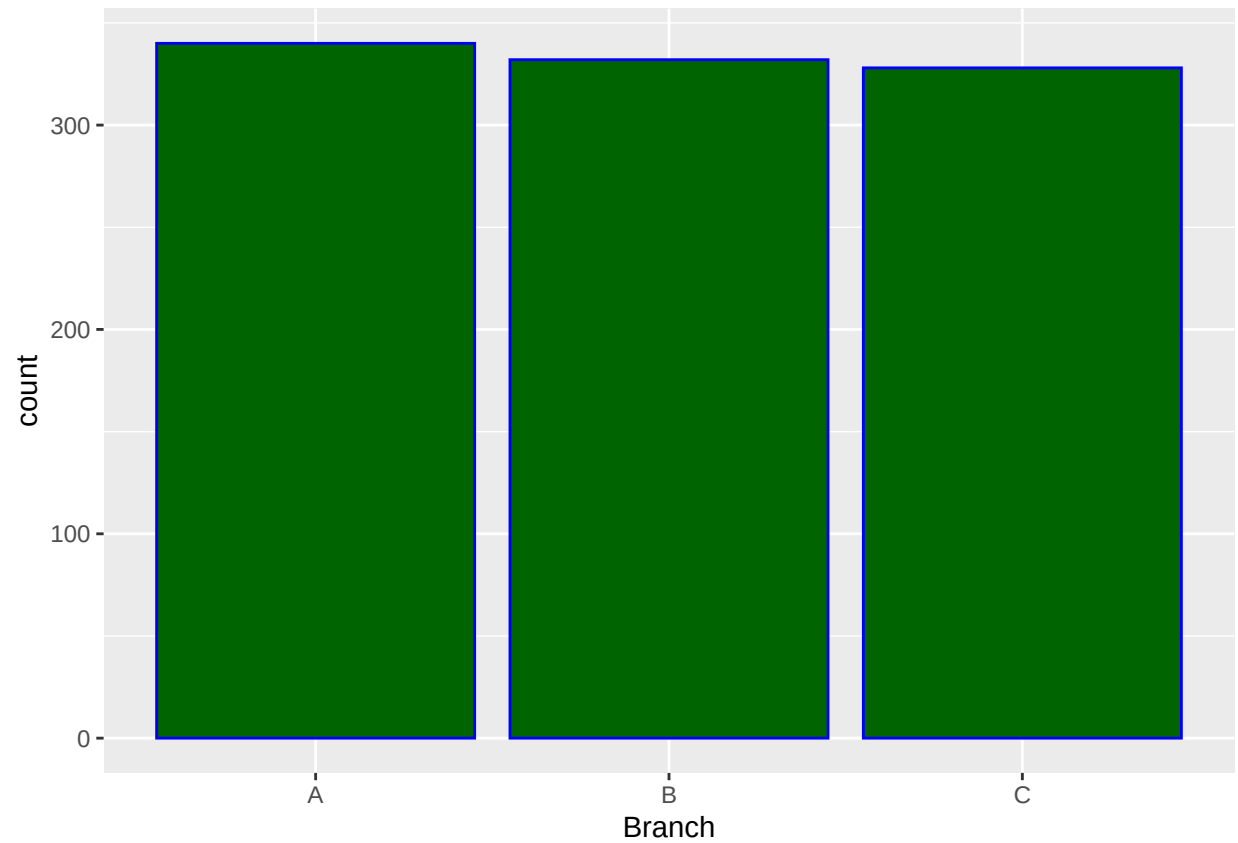




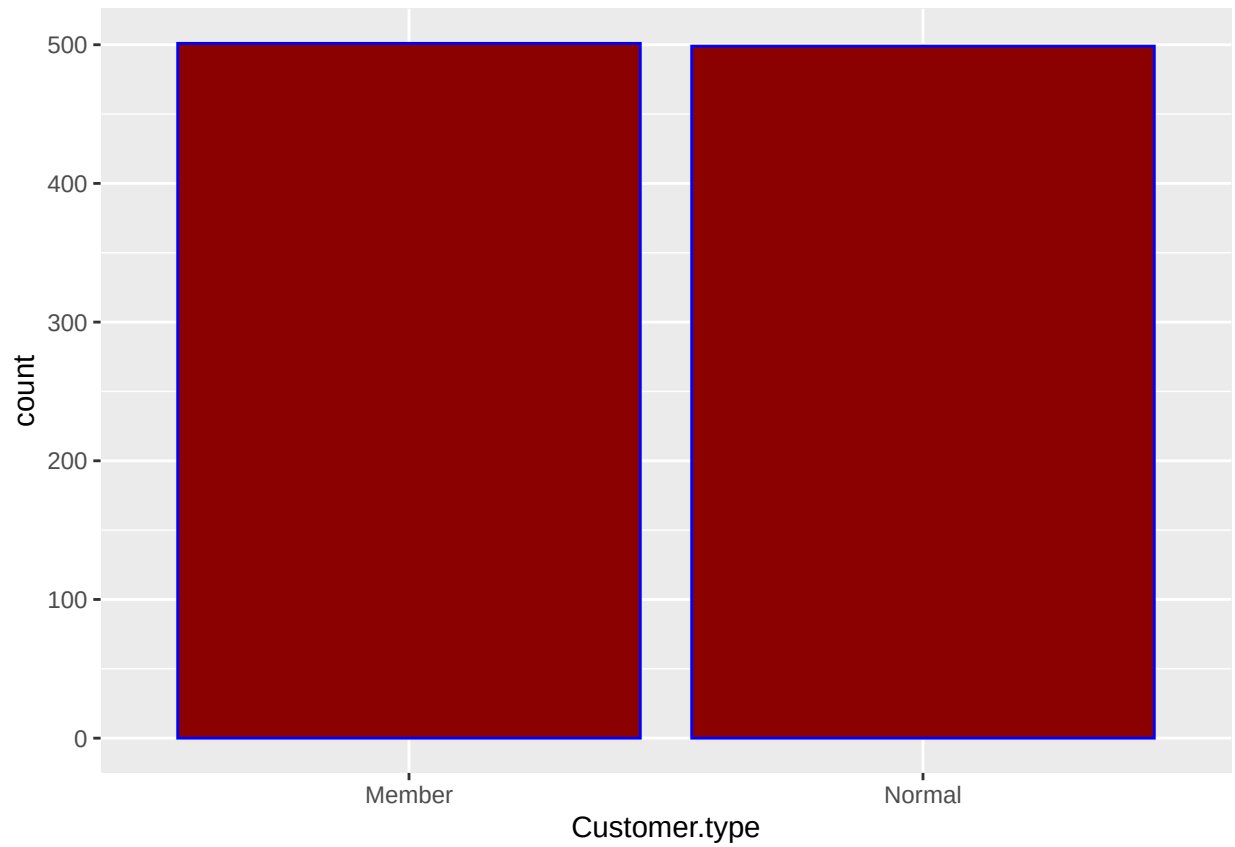


We do not have extreme outliers in our dataset

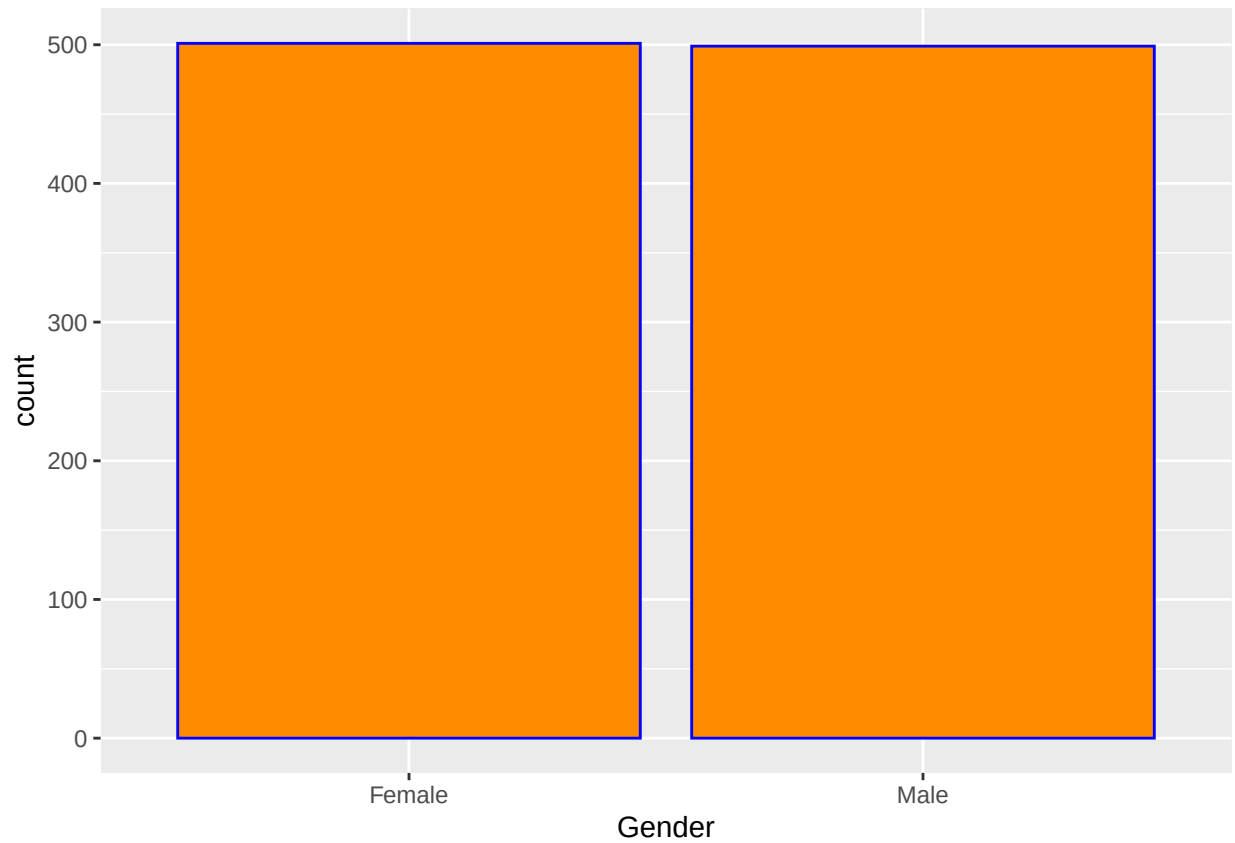
```
# let us visualize the Branch column  
ggplot(df, aes(x=Branch)) +  
  geom_bar(color="blue", fill="darkgreen" )
```



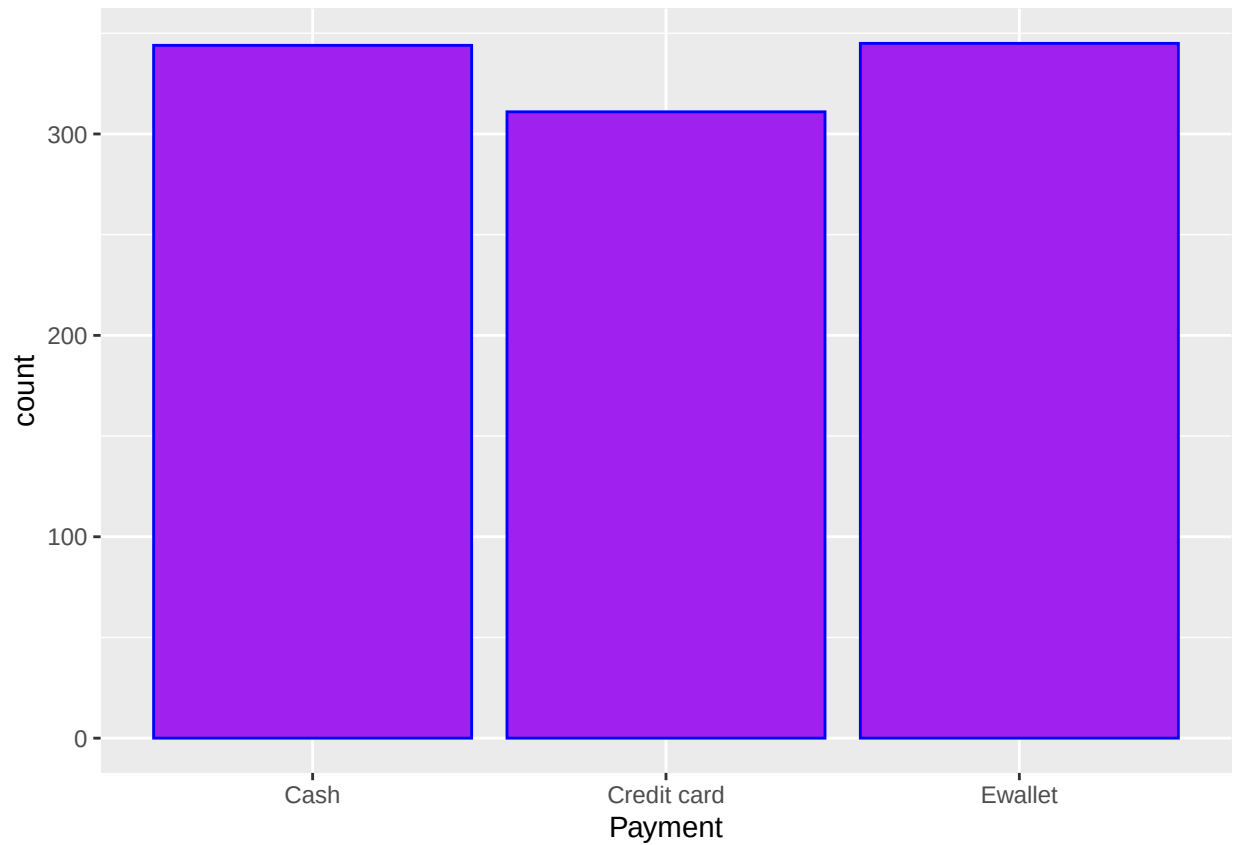
```
# let us see when the distributions of Customers at Carrefour  
ggplot(df, aes(x=Customer.type)) +  
  geom_bar(color="blue", fill="darkred" )
```



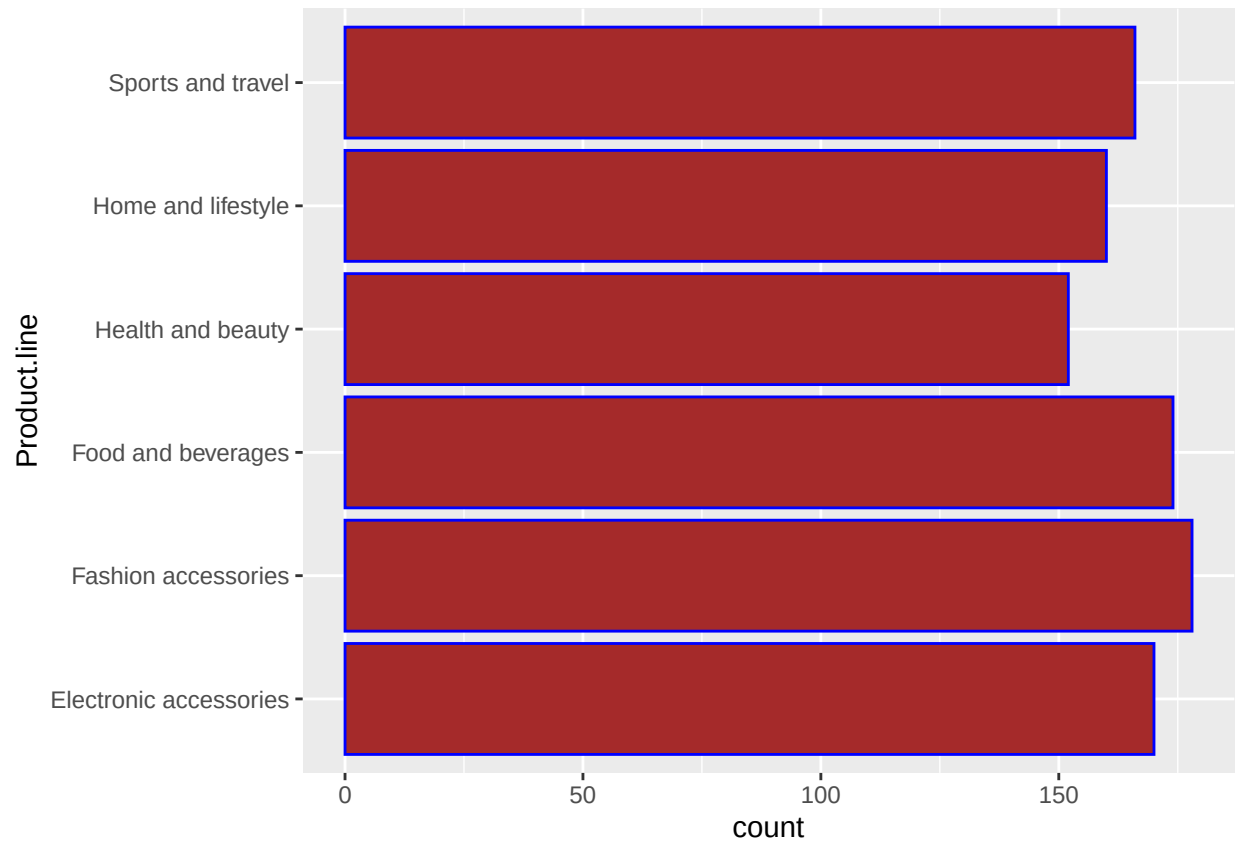
```
# let us see whethere a site visit earns revenue or not  
ggplot(df, aes(x=Gender)) +  
  geom_bar(color="blue", fill="darkorange")
```



```
# let us see which month has the most traffic  
ggplot(df, aes(x=Payment)) +  
  geom_bar(color="blue", fill="purple")
```

```
# let us see the distribution of the browser column  
ggplot(df, aes(y=Product.line))+  
  geom_bar(color="blue", fill="brown")
```



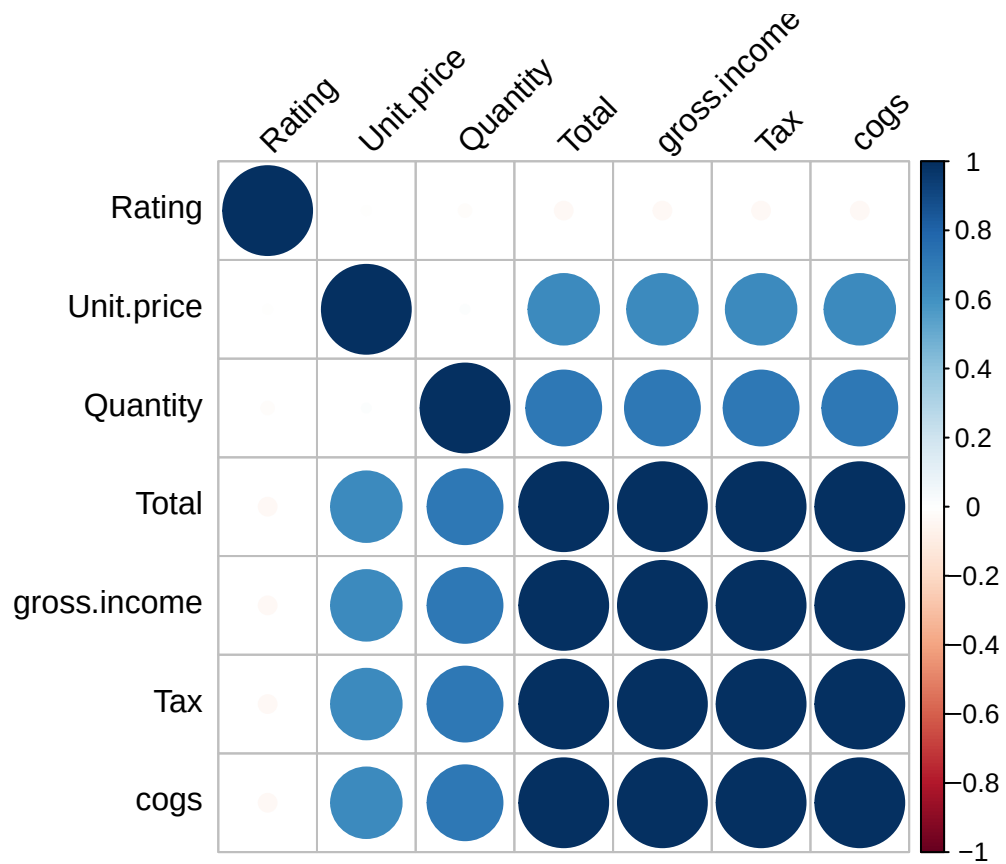
Bivariate and Multivariate Analysis

```
# find the correlations and round them off to 2 decimal places  
res <- round(cor(nums), 2)
```

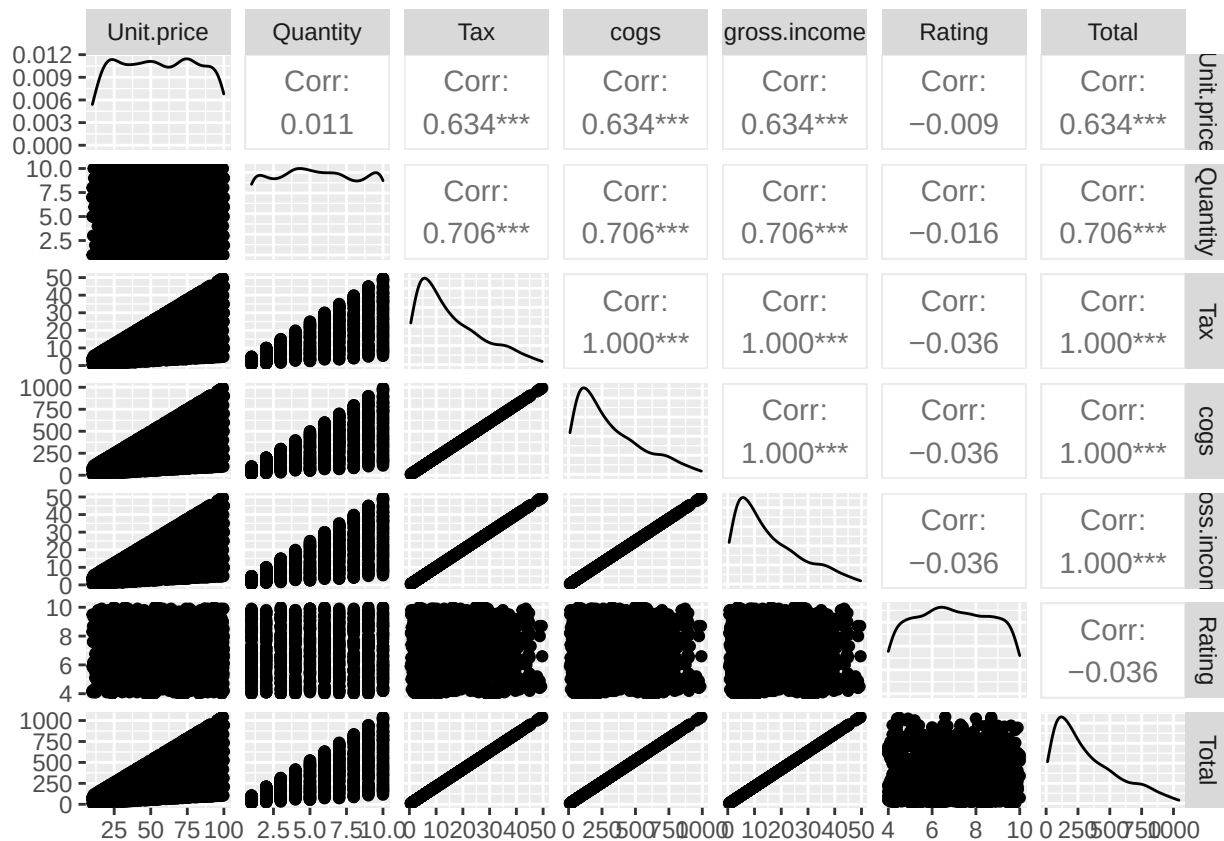
```
library (corrplot)
```

```
## corrplot 0.84 loaded
```

```
## corrplot 0.84 loaded  
corrplot(res, type = "full", order = "hclust", tl.col = "black", tl.srt = 45)
```



```
# let us make a pairplot
ggpairs(nums)
```



Part 1 - Implementing PCA

Data Preprocessing

Dealing with Outliers

```
# make a copy of the dataset
data<- df
names(nums)
```

```
## [1] "Unit.price"  "Quantity"    "Tax"         "cogs"        "gross.income"
## [6] "Rating"     "Total"
```

```
# let us remove the outliers since PCA is sensitive to outliers
# Tax
qnt <- quantile(data$Tax, probs=c(.25, .75), na.rm = T)
caps <- quantile(data$Tax, probs=c(.05, .95), na.rm = T)
H <- 1.5 * IQR(data$Tax, na.rm = T)
data$Tax[data$Tax < (qnt[1] - H)] <- caps[1]
data$Tax[data$Tax > (qnt[2] + H)] <- caps[2]

# cogs
```

```

qnt <- quantile(data$cogs, probs=c(.25, .75), na.rm = T)
caps <- quantile(data$cogs, probs=c(.05, .95), na.rm = T)
H <- 1.5 * IQR(data$cogs, na.rm = T)
data$cogs[data$cogs < (qnt[1] - H)] <- caps[1]
data$cogs[data$cogs > (qnt[2] + H)] <- caps[2]

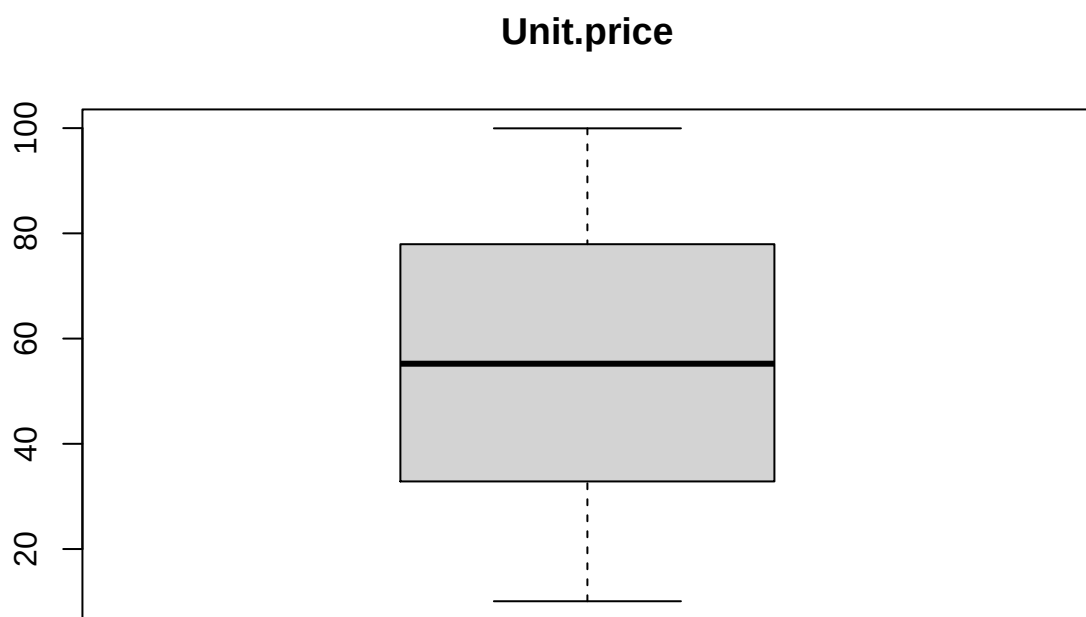
# gross income
qnt <- quantile(data$gross.income, probs=c(.25, .75), na.rm = T)
caps <- quantile(data$gross.income, probs=c(.05, .95), na.rm = T)
H <- 1.5 * IQR(data$gross.income, na.rm = T)
data$gross.income[data$gross.income < (qnt[1] - H)] <- caps[1]
data$gross.income[data$gross.income > (qnt[2] + H)] <- caps[2]

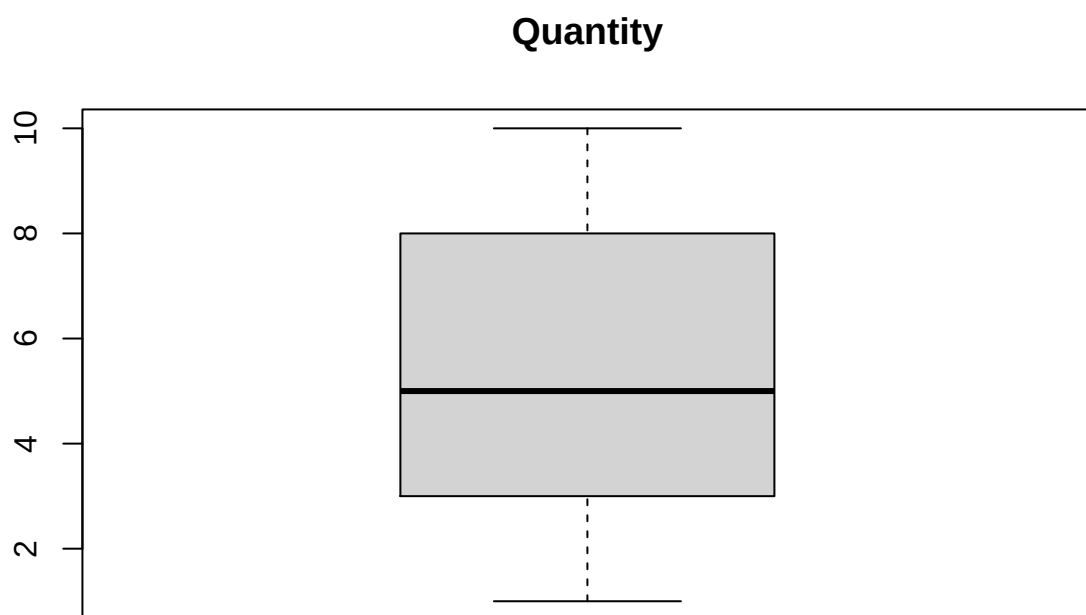
# Total
qnt <- quantile(data$Total, probs=c(.25, .75), na.rm = T)
caps <- quantile(data$Total, probs=c(.05, .95), na.rm = T)
H <- 1.5 * IQR(data$Total, na.rm = T)
data$Total[data$Total < (qnt[1] - H)] <- caps[1]
data$Total[data$Total > (qnt[2] + H)] <- caps[2]

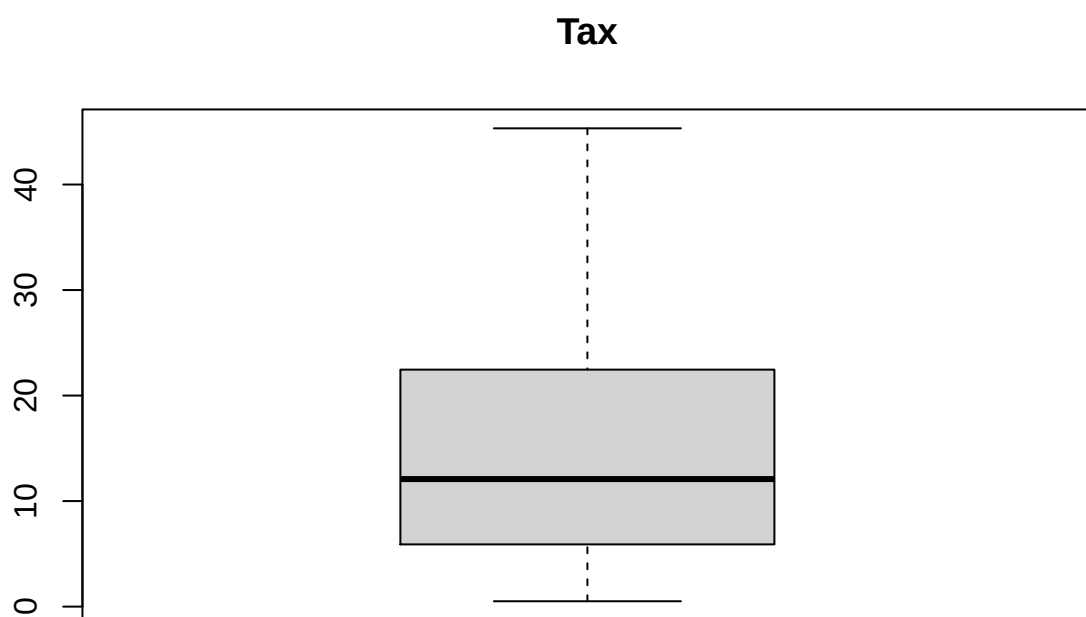
# let us make sure our columns have no extreme outliers
nums1 <- data %>% select_if(is.numeric)

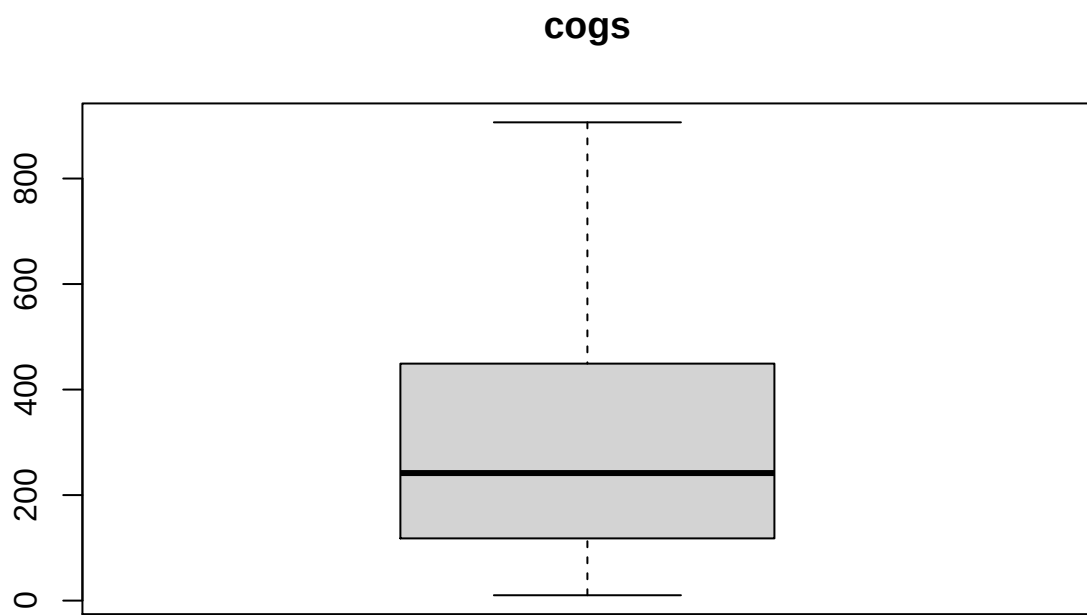
for (i in 1 : length (nums1)) {
  boxplot (nums1[,i], main= names (nums1[i]), type= "l" )
}

```

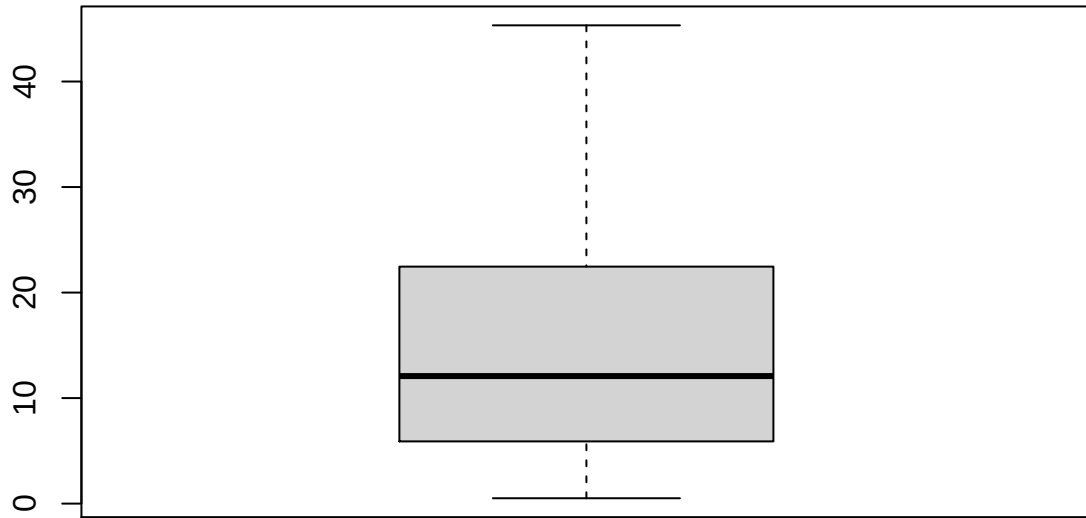


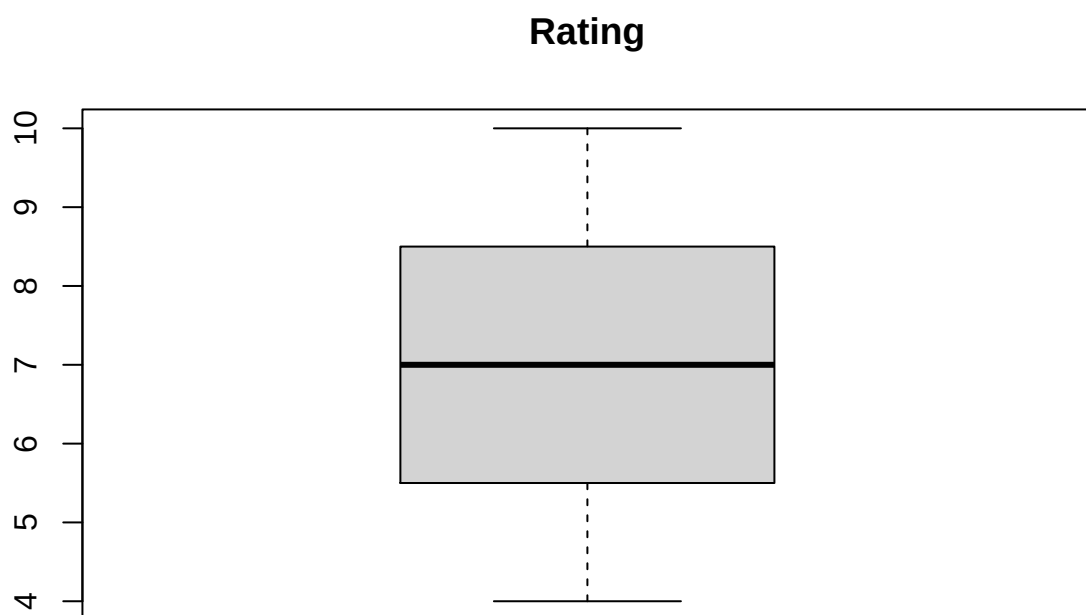


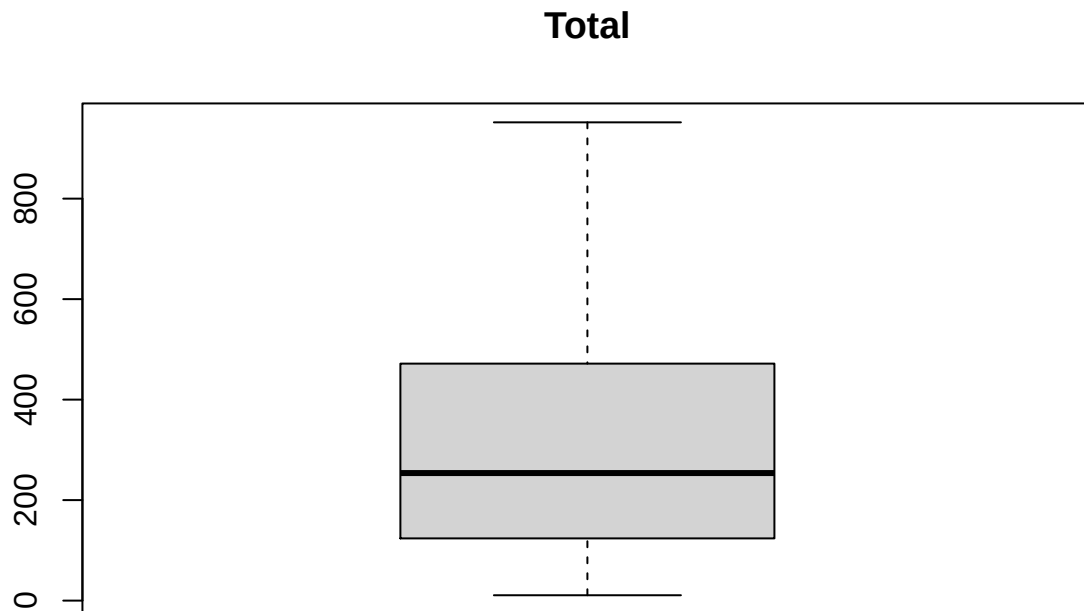




gross.income







```
str(data)
```

```
## 'data.frame':    1000 obs. of  15 variables:
## $ Invoice.ID      : chr  "750-67-8428" "226-31-3081" "631-41-3108" "123-19-1176" ...
## $ Branch         : chr  "A" "C" "A" "A" ...
## $ Customer.type   : chr  "Member" "Normal" "Normal" "Member" ...
## $ Gender          : chr  "Female" "Female" "Male" "Male" ...
## $ Product.line    : chr  "Health and beauty" "Electronic accessories" "Home and lifestyle" "Health and
## $ Unit.price      : num  74.7 15.3 46.3 58.2 86.3 ...
## $ Quantity        : int   7 5 7 8 7 7 6 10 2 3 ...
## $ Tax             : num   26.14 3.82 16.22 23.29 30.21 ...
## $ Date            : chr   "1/5/2019" "3/8/2019" "3/3/2019" "1/27/2019" ...
## $ Time            : chr   "13:08" "10:29" "13:23" "20:33" ...
## $ Payment         : chr   "Ewallet" "Cash" "Credit card" "Ewallet" ...
## $ cogs            : num   522.8 76.4 324.3 465.8 604.2 ...
## $ gross.income    : num   26.14 3.82 16.22 23.29 30.21 ...
## $ Rating          : num   9.1 9.6 7.4 8.4 5.3 4.1 5.8 8 7.2 5.9 ...
## $ Total           : num   549 80.2 340.5 489 634.4 ...
```

```
#merging the date and time column_to_rownames
```

```
library(anytime)
```

```
data$Timestamp <- with(data, anytime(paste(Date, Time)))
str(data)
```

```
## 'data.frame':    1000 obs. of  16 variables:
```

```
## $ Invoice.ID : chr "750-67-8428" "226-31-3081" "631-41-3108" "123-19-1176" ...
## $ Branch : chr "A" "C" "A" "A" ...
## $ Customer.type: chr "Member" "Normal" "Normal" "Member" ...
## $ Gender : chr "Female" "Female" "Male" "Male" ...
## $ Product.line : chr "Health and beauty" "Electronic accessories" "Home and lifestyle" "Health and
## $ Unit.price : num 74.7 15.3 46.3 58.2 86.3 ...
## $ Quantity : int 7 5 7 8 7 7 6 10 2 3 ...
## $ Tax : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Date : chr "1/5/2019" "3/8/2019" "3/3/2019" "1/27/2019" ...
## $ Time : chr "13:08" "10:29" "13:23" "20:33" ...
## $ Payment : chr "Ewallet" "Cash" "Credit card" "Ewallet" ...
## $ cogs : num 522.8 76.4 324.3 465.8 604.2 ...
## $ gross.income : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Rating : num 9.1 9.6 7.4 8.4 5.3 4.1 5.8 8 7.2 5.9 ...
## $ Total : num 549 80.2 340.5 489 634.4 ...
## $ Timestamp : POSIXct, format: "2019-01-05 13:08:00" "2019-03-08 10:29:00" ...
```

```
# extract the year, month, day and hour from the timestamp column
data$year <- format(data$Timestamp, format="%Y")
data$month <- format(data$Timestamp, format="%m")
data$day <- format(data$Timestamp, format="%d")
data$hour <- format(data$Timestamp, format="%H")
```

```
#convert the new columns created to categorical values(factor)
data$year <- as.factor(data$year)
data$month <- as.factor(data$month)
data$day <- as.factor(data$day)
data$hour <- as.factor(data$hour)
#check the datatypes of the resulting dataframe
str(data)
```

```
## 'data.frame': 1000 obs. of 20 variables:
## $ Invoice.ID : chr "750-67-8428" "226-31-3081" "631-41-3108" "123-19-1176" ...
## $ Branch : chr "A" "C" "A" "A" ...
## $ Customer.type: chr "Member" "Normal" "Normal" "Member" ...
## $ Gender : chr "Female" "Female" "Male" "Male" ...
## $ Product.line : chr "Health and beauty" "Electronic accessories" "Home and lifestyle" "Health and
## $ Unit.price : num 74.7 15.3 46.3 58.2 86.3 ...
## $ Quantity : int 7 5 7 8 7 7 6 10 2 3 ...
## $ Tax : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Date : chr "1/5/2019" "3/8/2019" "3/3/2019" "1/27/2019" ...
## $ Time : chr "13:08" "10:29" "13:23" "20:33" ...
## $ Payment : chr "Ewallet" "Cash" "Credit card" "Ewallet" ...
## $ cogs : num 522.8 76.4 324.3 465.8 604.2 ...
## $ gross.income : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Rating : num 9.1 9.6 7.4 8.4 5.3 4.1 5.8 8 7.2 5.9 ...
## $ Total : num 549 80.2 340.5 489 634.4 ...
## $ Timestamp : POSIXct, format: "2019-01-05 13:08:00" "2019-03-08 10:29:00" ...
## $ year : Factor w/ 1 level "2019": 1 1 1 1 1 1 1 1 1 1 ...
## $ month : Factor w/ 3 levels "01","02","03": 1 3 3 1 2 3 2 2 1 2 ...
## $ day : Factor w/ 31 levels "01","02","03",...: 5 8 3 27 8 25 25 24 10 20 ...
## $ hour : Factor w/ 11 levels "10","11","12",...: 4 1 4 11 1 9 5 2 8 4 ...
```

```
# let us now drop the timestamp, date and Time and Invoice column since we don't need them anymore
data$Timestamp <- NULL
data$Invoice.ID <- NULL
data$Time <- NULL
data$Date <- NULL
data$ year <- NULL
```

```
str(data)
```

```
## 'data.frame': 1000 obs. of 15 variables:
## $ Branch : chr "A" "C" "A" "A" ...
## $ Customer.type: chr "Member" "Normal" "Normal" "Member" ...
## $ Gender : chr "Female" "Female" "Male" "Male" ...
## $ Product.line : chr "Health and beauty" "Electronic accessories" "Home and lifestyle" "Health and
## $ Unit.price : num 74.7 15.3 46.3 58.2 86.3 ...
## $ Quantity : int 7 5 7 8 7 7 6 10 2 3 ...
## $ Tax : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Payment : chr "Ewallet" "Cash" "Credit card" "Ewallet" ...
## $ cogs : num 522.8 76.4 324.3 465.8 604.2 ...
## $ gross.income : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Rating : num 9.1 9.6 7.4 8.4 5.3 4.1 5.8 8 7.2 5.9 ...
## $ Total : num 549 80.2 340.5 489 634.4 ...
## $ month : Factor w/ 3 levels "01","02","03": 1 3 3 1 2 3 2 2 1 2 ...
## $ day : Factor w/ 31 levels "01","02","03",...: 5 8 3 27 8 25 25 24 10 20 ...
## $ hour : Factor w/ 11 levels "10","11","12",...: 4 1 4 11 1 9 5 2 8 4 ...
```

```
# convert data types
data$Branch <- as.factor(data$Branch)
data$Customer.type <- as.factor(data$Customer.type)
data$Gender <- as.factor(data$Gender)
data$Product.line <- as.factor(data$Product.line)
data$Payment <- as.factor(data$Payment)
data$Quantity <- as.numeric(data$Quantity)
data$month <- as.numeric(data$month)
data$day <- as.numeric(data$day)
data$hour <- as.numeric(data$hour)
```

```
# label encoding categorical values
# One hot encoding of the factor variables.
dmy <- dummyVars(" ~ .", data = data)
dummy_df <- data.frame(predict(dmy, newdata = data))
#print(dummy_df)
str(dummy_df)
```

```
## 'data.frame': 1000 obs. of 26 variables:
## $ Branch.A : num 1 0 1 1 1 0 1 0 1 0 ...
## $ Branch.B : num 0 0 0 0 0 0 0 0 0 1 ...
## $ Branch.C : num 0 1 0 0 0 1 0 1 0 0 ...
## $ Customer.type.Member : num 1 0 0 1 0 0 1 0 1 1 ...
## $ Customer.type.Normal : num 0 1 1 0 1 1 0 1 0 0 ...
## $ Gender.Female : num 1 1 0 0 0 0 1 1 1 1 ...
## $ Gender.Male : num 0 0 1 1 1 1 0 0 0 0 ...
```

```
## $ Product.line.Electronic.accessories: num 0 1 0 0 0 1 1 0 0 0 ...
## $ Product.line.Fashion.accessories : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Product.line.Food.and.beverages : num 0 0 0 0 0 0 0 0 0 1 ...
## $ Product.line.Health.and.beauty : num 1 0 0 1 0 0 0 0 1 0 ...
## $ Product.line.Home.and.lifestyle : num 0 0 1 0 0 0 0 1 0 0 ...
## $ Product.line.Sports.and.travel : num 0 0 0 0 1 0 0 0 0 0 ...
## $ Unit.price : num 74.7 15.3 46.3 58.2 86.3 ...
## $ Quantity : num 7 5 7 8 7 7 6 10 2 3 ...
## $ Tax : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Payment.Cash : num 0 1 0 0 0 0 0 0 0 0 ...
## $ Payment.Credit.card : num 0 0 1 0 0 0 0 0 1 1 ...
## $ Payment.Ewallet : num 1 0 0 1 1 1 1 1 0 0 ...
## $ cogs : num 522.8 76.4 324.3 465.8 604.2 ...
## $ gross.income : num 26.14 3.82 16.22 23.29 30.21 ...
## $ Rating : num 9.1 9.6 7.4 8.4 5.3 4.1 5.8 8 7.2 5.9 ...
## $ Total : num 549 80.2 340.5 489 634.4 ...
## $ month : num 1 3 3 1 2 3 2 2 1 2 ...
## $ day : num 5 8 3 27 8 25 25 24 10 20 ...
## $ hour : num 4 1 4 11 1 9 5 2 8 4 ...
```

Now that we have label encoded our categorical features and created new features as well as eliminated some variables, we can apply PCA to our Dataset

PCA is a method of obtaining important variables (in form of components) from a large set of variables available in a data set. It extracts low dimensional set of features by taking a projection of irrelevant dimensions from a high dimensional data set with a motive to capture as much information as possible. With fewer variables obtained while minimizing the loss of information, visualization also becomes much more meaningful. PCA is more useful when dealing with 3 or higher dimensional data.

```
#removing the dependent (response) variable and other identifier variables( if any)
# our total price is the predicted variable
data <- dummy_df
my_data <- subset(data, select = -c(Total))
#check available variables
colnames(my_data)
```

```
## [1] "Branch.A" "Branch.B"
## [3] "Branch.C" "Customer.type.Member"
## [5] "Customer.type.Normal" "Gender.Female"
## [7] "Gender.Male" "Product.line.Electronic.accessories"
## [9] "Product.line.Fashion.accessories" "Product.line.Food.and.beverages"
## [11] "Product.line.Health.and.beauty" "Product.line.Home.and.lifestyle"
## [13] "Product.line.Sports.and.travel" "Unit.price"
## [15] "Quantity" "Tax"
## [17] "Payment.Cash" "Payment.Credit.card"
## [19] "Payment.Ewallet" "cogs"
## [21] "gross.income" "Rating"
## [23] "month" "day"
## [25] "hour"
```

```
# We pass the dataframe to the prcomp(). We also set two arguments, center and scale,
# to be TRUE then preview our object with summary
my_data.pca <- prcomp(my_data, center = TRUE, scale. = TRUE)
summary(my_data.pca)
```

```
## Importance of components:
##          PC1      PC2      PC3      PC4      PC5      PC6      PC7
## Standard deviation    1.987 1.46598 1.39985 1.26400 1.24720 1.22158 1.20507
## Proportion of Variance 0.158 0.08596 0.07838 0.06391 0.06222 0.05969 0.05809
## Cumulative Proportion 0.158 0.24391 0.32229 0.38620 0.44842 0.50811 0.56620
##          PC8      PC9      PC10     PC11     PC12     PC13     PC14
## Standard deviation    1.11673 1.10501 1.09284 1.08750 1.07169 1.04417 1.0024
## Proportion of Variance 0.04988 0.04884 0.04777 0.04731 0.04594 0.04361 0.0402
## Cumulative Proportion 0.61608 0.66493 0.71270 0.76000 0.80594 0.84956 0.8898
##          PC15     PC16     PC17     PC18     PC19     PC20
## Standard deviation    0.97180 0.95332 0.90573 0.28749 5.741e-15 4.832e-15
## Proportion of Variance 0.03778 0.03635 0.03281 0.00331 0.000e+00 0.000e+00
## Cumulative Proportion 0.92753 0.96388 0.99669 1.00000 1.000e+00 1.000e+00
##          PC21     PC22     PC23     PC24     PC25
## Standard deviation    2.777e-15 2.425e-15 1.372e-15 3.29e-16 1.576e-16
## Proportion of Variance 0.000e+00 0.000e+00 0.000e+00 0.00e+00 0.000e+00
## Cumulative Proportion 1.000e+00 1.000e+00 1.000e+00 1.00e+00 1.000e+00
```

As a result we obtain 15 principal components, each which explain a percentate of the total variation of the dataset. PC1 explains 28% of the total variance.. PC2 explains 8%% of the variance and so on.

```
# Calling str() to have a look at your PCA object
str(my_data.pca)
```

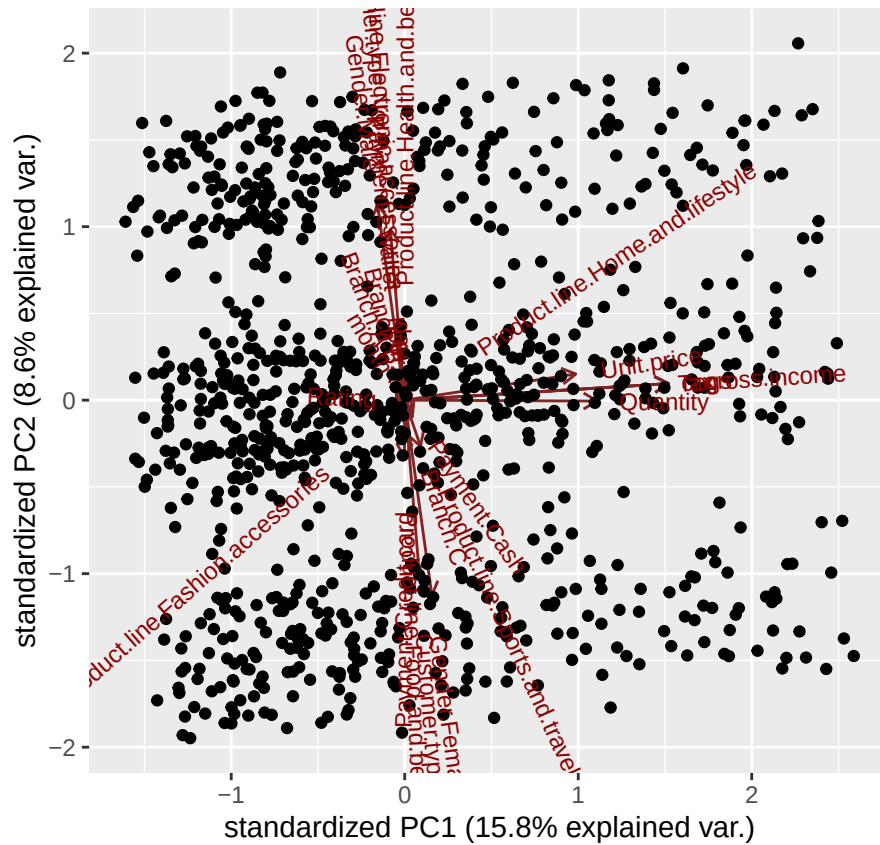
```
## List of 5
## $ sdev      : num [1:25] 1.99 1.47 1.4 1.26 1.25 ...
## $ rotation: num [1:25, 1:25] -0.02369 -0.00607 0.02999 0.02727 -0.02727 ...
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : chr [1:25] "Branch.A" "Branch.B" "Branch.C" "Customer.type.Member" ...
##   .. ..$ : chr [1:25] "PC1" "PC2" "PC3" "PC4" ...
## $ center    : Named num [1:25] 0.34 0.332 0.328 0.501 0.499 0.501 0.499 0.17 0.178 0.174 ...
##   ..- attr(*, "names")= chr [1:25] "Branch.A" "Branch.B" "Branch.C" "Customer.type.Member" ...
## $ scale     : Named num [1:25] 0.474 0.471 0.47 0.5 0.5 ...
##   ..- attr(*, "names")= chr [1:25] "Branch.A" "Branch.B" "Branch.C" "Customer.type.Member" ...
## $ x         : num [1:1000, 1:25] 1.9151 -1.96379 -0.00143 1.25341 2.34181 ...
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : chr [1:1000] "1" "2" "3" "4" ...
##   .. ..$ : chr [1:25] "PC1" "PC2" "PC3" "PC4" ...
## - attr(*, "class")= chr "prcomp"
```

Here we note that our pca object: The center point (*center*), *scaling*(scale), standard deviation(sdev) of each principal component. The relationship (correlation or anticorrelation, etc) between the initial variables and the principal components (*rotation*).*The values of each sample in terms of the principal components(x)*

```
dim(my_data.pca$x)
```

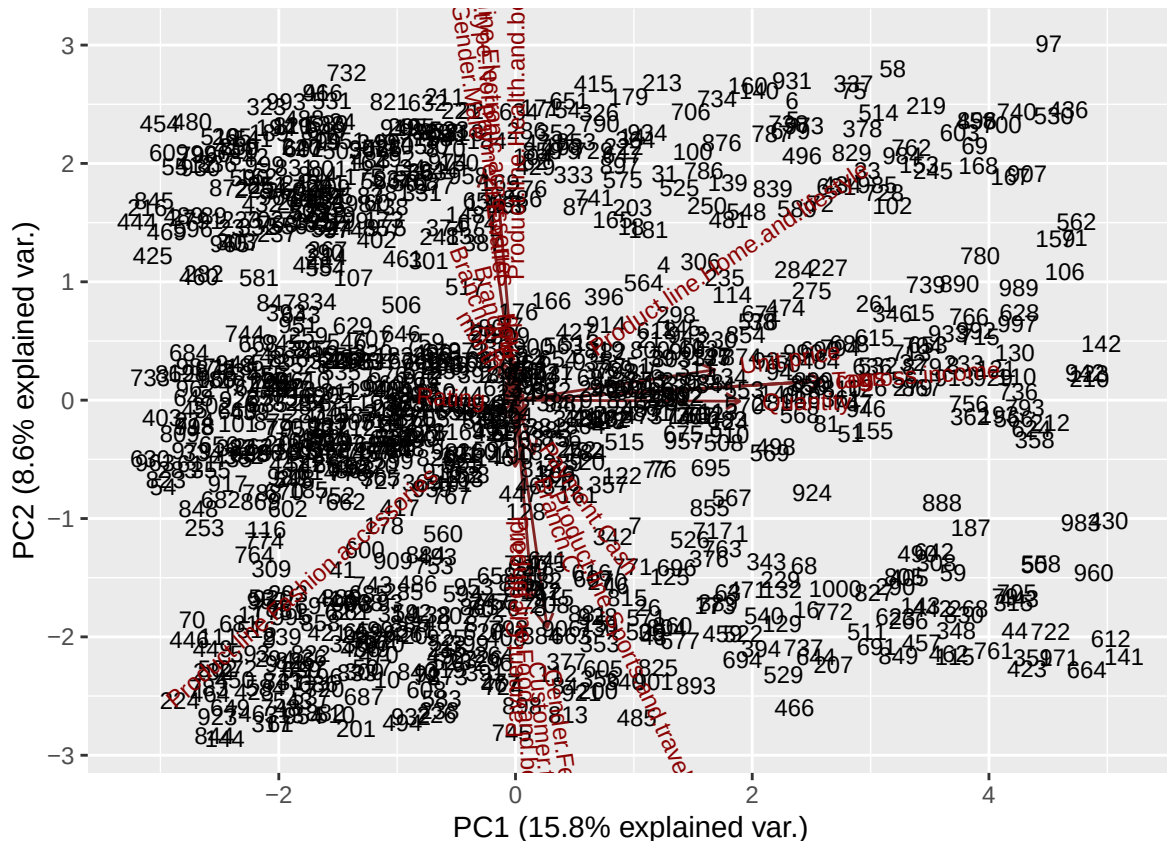
```
## [1] 1000 25
```

```
# We will now plot our pca. This will provide us with some very useful insights
ggbiplot(my_data.pca)
```

We can see that Unit price, Gross income and Quantity contribute to PC1

```
# Adding more detail to the plot, we provide arguments rownames as labels
ggbiplot(my_data.pca, labels=rownames(my_data), obs.scale = 1, var.scale = 1)
```



PC1 (15.8% explained var.)

```
#compute standard deviation of each principal component
```

```
std_dev <- my_data.pca$sdev
```

```
#compute variance
```

```
pr_var <- std_dev^2
```

We aim to find the components which explain the maximum variance. This is because, we want to retain

#To compute the proportion of variance explained by each component, we simply divide the variance by sum

#proportion of variance explained

```
prop_varex <- pr_var/sum(pr_var)
```

```
prop_varex[1:10]
```

```
## [1] 0.15794575 0.08596417 0.07838374 0.06390790 0.06222037 0.05969073
```

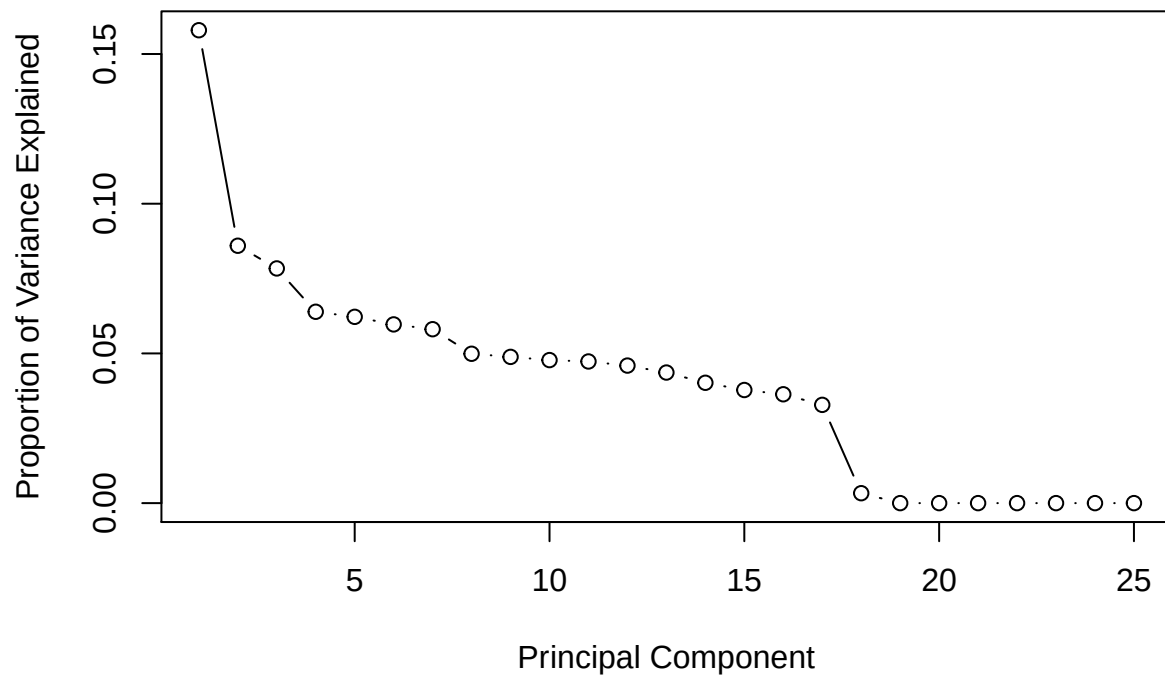
```
## [7] 0.05808789 0.04988350 0.04884184 0.04777176
```

This shows that first principal component explains 15.7% variance. Second component explains 8.5% variance. Third component explains 7.8% variance and so on.

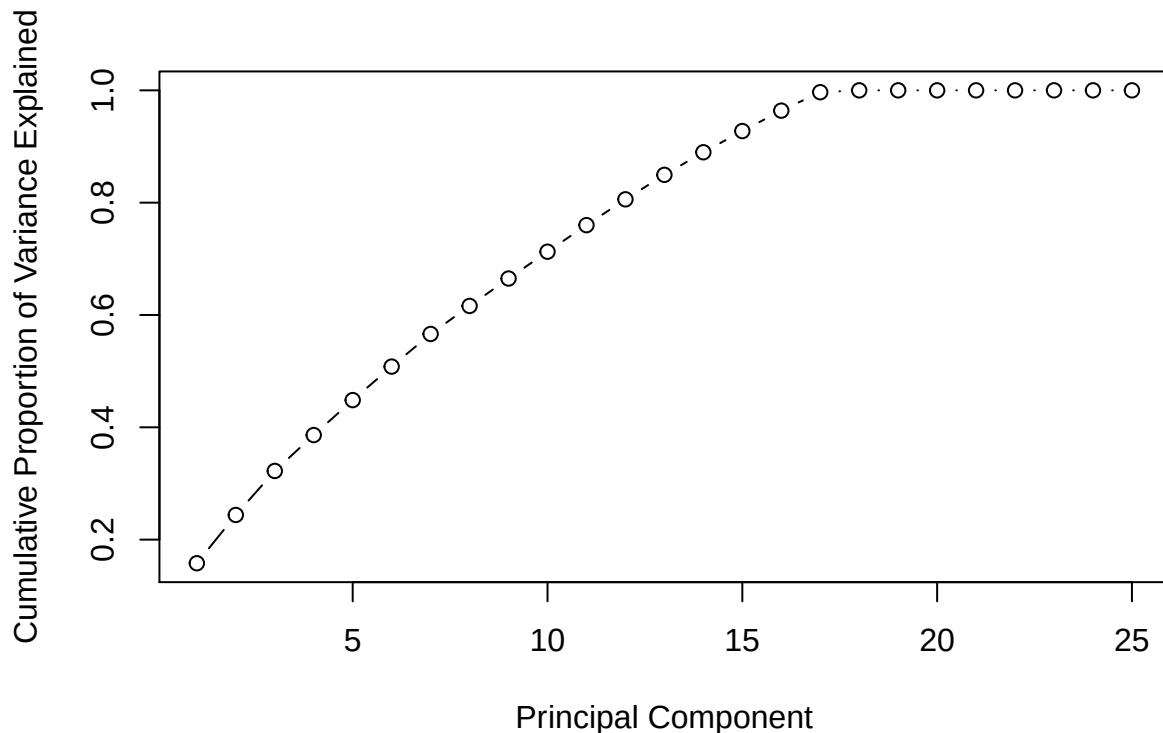
Now that we have all these components that contribute to our sales, how do we decide how many components should be selected for modeling stage?

The answer to this question is provided by a scree plot. A scree plot is used to access components or factors which explains the most of variability in the data. It represents values in descending order.

```
#scree plot
plot(prop_varex, xlab = "Principal Component",
      ylab = "Proportion of Variance Explained",
      type = "b")
```



```
#cumulative scree plot to give us clarity
plot(cumsum(prop_varex), xlab = "Principal Component",
      ylab = "Cumulative Proportion of Variance Explained",
      type = "b")
```



From the above plots we can see that 17 components explain above 98% of the variance in the data set. In other words, using PCA we have reduced 25 predictors to 17 without compromising on explained variance. This is the power of PCA.

Part 2: Feature Selection

Feature selection is the process of selecting a subset of relevant features (variables, predictors) for use in model construction. This becomes even more important when the number of features are very large. We can use the `findCorrelation` function included in the `caret` package to create a subset of variables. This function would allow us to remove redundancy by correlation using the given dataset. It would search through a correlation matrix and return a vector of integers corresponding to the columns to remove or reduce pair-wise correlations.

```
library(corrplot)
# Calculating the correlation matrix
correlationMatrix <- cor(my_data)

# Find attributes that are highly correlated
highlyCorrelated <- findCorrelation(correlationMatrix, cutoff=0.75)

# Highly correlated attributes
highlyCorrelated
```

```
## [1] 16 20 6 4
```

```
names(my_data[,highlyCorrelated])
```

```
## [1] "Tax" "cogs" "Gender.Female"
## [4] "Customer.type.Member"
```

```
# We can remove the variables with a higher correlation
# and comparing the results graphically as shown below
# Removing Redundant Features
```

```
Dataset2<-my_data[-highlyCorrelated]
```

```
colnames(Dataset2)
```

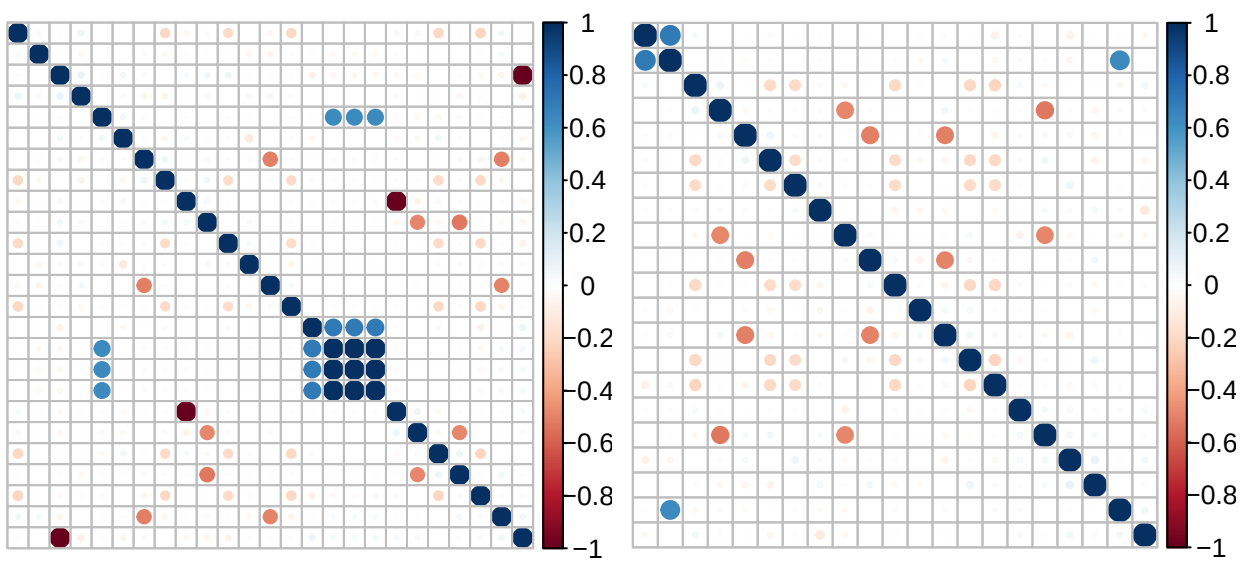
```
## [1] "Branch.A" "Branch.B"
## [3] "Branch.C" "Customer.type.Normal"
## [5] "Gender.Male" "Product.line.Electronic.accessories"
## [7] "Product.line.Fashion.accessories" "Product.line.Food.and.beverages"
## [9] "Product.line.Health.and.beauty" "Product.line.Home.and.lifestyle"
## [11] "Product.line.Sports.and.travel" "Unit.price"
## [13] "Quantity" "Payment.Cash"
## [15] "Payment.Credit.card" "Payment.Ewallet"
## [17] "gross.income" "Rating"
## [19] "month" "day"
## [21] "hour"
```

```
# Performing our graphical comparison
```

```
par(mfrow = c(1, 2))
```

```
corrplot(correlationMatrix, order = "hclust", tl.pos='n')
```

```
corrplot(cor(Dataset2), order = "hclust", tl.pos='n')
```



Follow up Questions Did we have the right data? Yes we did

Did we have the right question? Yes we did

Do we need to do anything else to answer the question? No